MIRROR: Reflecting Hidden Symmetries from Offline Robot Interaction Data

Shiqi Liu[1] **Yihang Yao**[1] **Peide Huang**[1] **Mengdi Xu**[2] **Ding Zhao**[1]
[1]Carnegie Mellon University [2]Stanford University

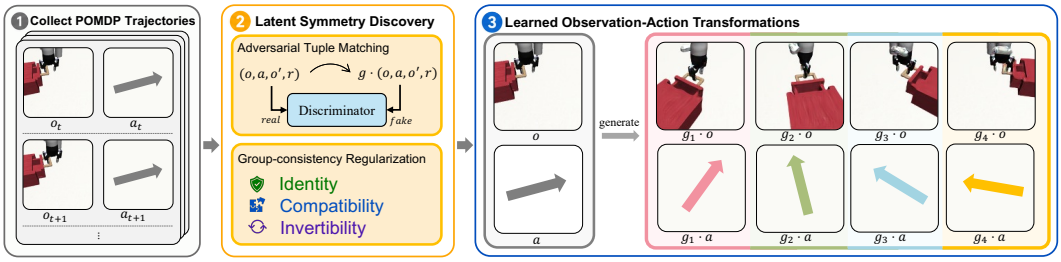


Figure 1: **Overview of MIRROR.** From offline robot interaction data, MIRROR learns how a specified symmetry group jointly transforms observations and actions. Adversarial matching encourages transformed transition tuples to preserve the original data distribution, while group-consistency regularization encourages identity, composition, and inverse consistency. The examples on the right illustrate the learned coupling: transforming a scene also transforms its associated robot command, mapping an observation–action pair (o, a) to corresponding pairs $(g \cdot o, g \cdot a)$.

Abstract: Symmetry-aware robot policy learning requires coupled observation–action transformations. For pixel observations, even simple physical symmetries can induce complex, viewpoint-dependent image transformations that are difficult to specify by hand. Existing symmetry-aware methods often rely on representations with predefined transformations, requiring preprocessing tailored to the sensing setup and task. We introduce MIRROR, a framework that learns how a known symmetry group operates jointly on observations and actions in partially observable Markov decision processes (POMDPs) from offline interaction data. MIRROR couples these transformations through adversarial tuple matching over one-step transition tuples, while explicit group-consistency regularization penalizes violations of the group-operation axioms. On four Robosuite tabletop manipulation tasks, MIRROR achieves the lowest aggregate error on three of four symmetry-quality metrics, including a $\mathbf{61}\%$ reduction in compatibility error—which measures consistency under transformation composition—relative to an adapted LaLiGAN baseline. Using the learned transformations in frame-averaged behavior cloning (BC) yields higher mean success rates than vanilla BC on all four tasks, increasing average success from 80.48% to 85.12%, with per-task gains up to $\mathbf{9.66}$ percentage points. We also repeat symmetry discovery and policy training in simulation with heightmap and segmentation-mask observations, then deploy the resulting policy zero-shot on a real Kinova GEN3 arm. Project page: https://pomdp-mirror.github.io.

Keywords: Symmetry Discovery, Equivariant Policy Learning, Robot Manipulation

1 Introduction

Symmetry can improve sample efficiency and generalization in robot policy learning by allowing experience to be shared across equivalent task configurations [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]. Exploiting this structure requires coupled observation–action transformations, which existing symmetry-aware methods typically assume are known. This assumption is difficult to satisfy when the robot observes the scene through images. For example, rotating a drawer changes both its appearance to

a fixed camera and the direction needed to pull it open. Although the physical rotation is simple, the resulting image transformation depends on the scene geometry and viewpoint and generally cannot be expressed as an image rotation. Existing methods address this difficulty by using point clouds [3, 4, 5, 6, 7, 8] or projecting observations into representations with known group operations [1, 2, 9, 10]. These representations make the group operation explicit, but constructing them can require preprocessing tailored to the sensing setup and task. Extending the same approach to a new observation modality can therefore require redesigning how observations are represented and transformed. Thus, even when the relevant symmetry group is known, exploiting it still depends on manually specifying suitable representations and the corresponding observation–action transformations.

This motivates a natural question: can we learn the required representations and coupled observation–action transformations directly from environment interaction data? Existing symmetry-discovery methods provide a promising foundation; for example, LaLiGAN [11] learns nonlinear symmetries of high-dimensional data through latent representations. Applying these approaches to visual control introduces two requirements. First, observations and actions must transform together to preserve their relationships with subsequent observations and rewards. Second, the learned transformations must satisfy the group-operation axioms: applying two transformations successively should agree with applying their composition. Distribution matching alone does not ensure this consistency, and even an exact linear group operation in latent space can lose it after approximate encoding and decoding. These requirements motivate learning coupled observation–action transformations while explicitly regularizing their group consistency.

We introduce **MIRROR**, a framework that learns how a known symmetry group G operates jointly on observations and actions from offline interaction data in partially observable Markov decision processes (POMDPs). The core idea is to match the distributions of original and transformed one-step transition tuples $\tau = (o, a, o', r)$, coupling observation and action learning through the observed transitions and rewards. We implement this distribution matching with an adversarial objective, which we call *adversarial tuple matching*. Identity, compatibility, and invertibility regularizers penalize violations of the group-operation axioms. To represent nonlinear transformations of pixel observations, MIRROR first compresses images with a pretrained, frozen perceptual autoencoder. Separate encoder–decoder pairs then map these perceptual latents and the actions to symmetry latent spaces with fixed linear representations of G. Both objectives operate on tuples of perceptual latents, actions, and rewards. Learning these mappings from each task's data allows the same discovery procedure to be used across tasks without manually specifying their coupled observation–action transformations.

We evaluate MIRROR on four Robosuite [12] tabletop manipulation tasks with RGB observations. It achieves the lowest aggregate error on three of four symmetry-quality metrics, including a 61% reduction in compatibility error relative to our adapted LaLiGAN [11] baseline. Incorporating the discovered transformations into frame-averaged behavior cloning (BC) yields higher mean success rates than vanilla BC on all four tasks. Average success increases from 80.48% to 85.12%, with the largest per-task gain of 9.66 percentage points on Cube Dropping. We also apply the framework to heightmap and segmentation-mask observations, repeating symmetry discovery and policy training in simulation before deploying the resulting policy zero-shot on a Kinova GEN3 arm.

Our contributions:

- **Adversarial symmetry discovery in POMDPs:** We formulate the discovery of coupled observation–action transformations as adversarial tuple matching over one-step transition tuples (o, a, o', r).
- **Explicit group-consistency regularization:** We penalize violations of identity, compatibility, and invertibility, improving aggregate compatibility and invertibility errors over the adapted GAN baselines.
- **Policy learning with discovered symmetries:** We use the learned transformations in a frame-averaged BC policy, improving simulation success rates by up to 9.66 percentage

points over vanilla BC. We also demonstrate the framework with a second observation modality and zero-shot policy deployment on a Kinova GEN3 arm.

2 Related Work

Exploiting symmetry in robot policy learning. Symmetry has been extensively studied as an inductive bias for robotic manipulation, yielding consistent gains in sample efficiency and generalization [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]. A central limitation is that these methods assume the way the symmetry group operates on observations and actions is known *a priori*, which rarely holds for visual manipulation tasks. Existing work circumvents this with task-specific observation preprocessing, such as using point clouds from depth cameras [3, 4, 5, 6, 7, 8] or projecting observations into a representation where the symmetry operates linearly. Concrete instances of the latter include projecting depth images to a top-down heightmap [1], inpainting images to make the top-down view equivariant [2], mapping fisheye images to a 3D sphere [9], and lifting 2D images to 3D ray features [10]. No single recipe transfers across environments. MIRROR addresses this limitation by learning coupled nonlinear observation–action transformations for a specified group G from offline interaction data, reducing dependence on manually designed, modality-specific transformations.

Data-driven symmetry discovery. Automatic symmetry discovery identifies unknown symmetries directly from data [13, 14, 15] and has improved downstream tasks such as equation discovery and long-horizon forecasting [16, 11]. However, two limitations restrict prior work's applicability to control in POMDPs. First, prior methods discover symmetries on observations alone, whereas decision-making requires *coupled* observation–action transformations that preserve the transition structure. Second, prior methods either omit explicit group-consistency regularization entirely [17, 18, 16] or regularize only a subset of the group-operation axioms, as in LaLiGAN [11], which regularizes only the identity axiom and accordingly violates the other group-operation axioms in practice. To address both gaps, we discover coupled observation–action transformations from one-step transition tuples $\tau = (o, a, o', r)$ via adversarial tuple matching, and we explicitly regularize the group-operation axioms (identity, compatibility, invertibility) to encourage the learned transformations to behave as valid group operations.

3 Preliminary

Partially Observable Markov Decision Processes. A POMDP $\mathcal{M} = (\mathcal{S}, \mathcal{A}, \mathcal{O}, P, \Omega, R, \mu_0)$ has measurable state, action, and observation spaces $\mathcal{S}, \mathcal{A}, \mathcal{O}$, respectively. Let $\mathcal{P}(\mathcal{X})$ denote the set of probability measures on $\mathcal{X}$. The maps $P : \mathcal{S} \times \mathcal{A} \to \mathcal{P}(\mathcal{S})$ and $\Omega : \mathcal{S} \to \mathcal{P}(\mathcal{O})$ are the transition and observation kernels, respectively. The initial state distribution is $\mu_0 \in \mathcal{P}(\mathcal{S})$, and $R : \mathcal{S} \times \mathcal{A} \to \mathbb{R}^{d_r}$ is a measurable d_r-dimensional reward function.

Groups, group operations, and representations. A *group* is a set G with an associative binary operation $\circ$, an identity element e, and inverses. A *group operation*[1] of G on a set $\mathcal{X}$ is a map $\phi : G \times \mathcal{X} \to \mathcal{X}$ satisfying three axioms: *identity*, $\phi(e, x) = x$; *compatibility*, $\phi(g_2, \phi(g_1, x)) = \phi(g_2 \circ g_1, x)$; and *invertibility*, $\phi(g^{-1}, \phi(g, x)) = x$; we sometimes write $g \cdot x := \phi(g, x)$. A *linear representation* of G on a vector space V is a homomorphism $\rho : G \to \mathrm{GL}(V)$ with $\rho(g_2 \circ g_1) = \rho(g_2)\rho(g_1)$ and $\rho(e) = I$.

Symmetric POMDPs. A POMDP is *G-symmetric* if measurable group operations ϕ_s, ϕ_a, ϕ_o on $\mathcal{S}, \mathcal{A}, \mathcal{O}$, respectively, make its transition and observation kernels equivariant and its reward invariant: $P(g{\cdot}B_s \mid g{\cdot}s, g{\cdot}a) = P(B_s \mid s, a)$, $\Omega(g{\cdot}B_o \mid g{\cdot}s) = \Omega(B_o \mid s)$, and $R(g{\cdot}s, g{\cdot}a) = R(s, a)$, for all $g \in G$, $s \in \mathcal{S}$, $a \in \mathcal{A}$. Invariance of the offline transition-tuple distribution additionally depends on data collection.

[1] We use the term *group operation* (rather than the more common *group action*) to avoid confusion with the *actions* in a POMDP.

Observable symmetry in POMDPs. Given group operations ϕ_o, ϕ_a of G on observations and actions, the induced transformation on one-step transition tuples $\tau = (o, a, o', r)$ is $\Phi(g, \tau) := \big(\phi_o(g, o),\ \phi_a(g, a),\ \phi_o(g, o'),\ r\big)$. The POMDP has *observable symmetry* under G if the resulting transition-tuple distribution is invariant: $p(\tau) = p(\Phi(g, \tau))$ for all $g \in G$, where $p(\tau)$ is the distribution sampled from the offline dataset $\mathcal{D}$.

4 Methodology

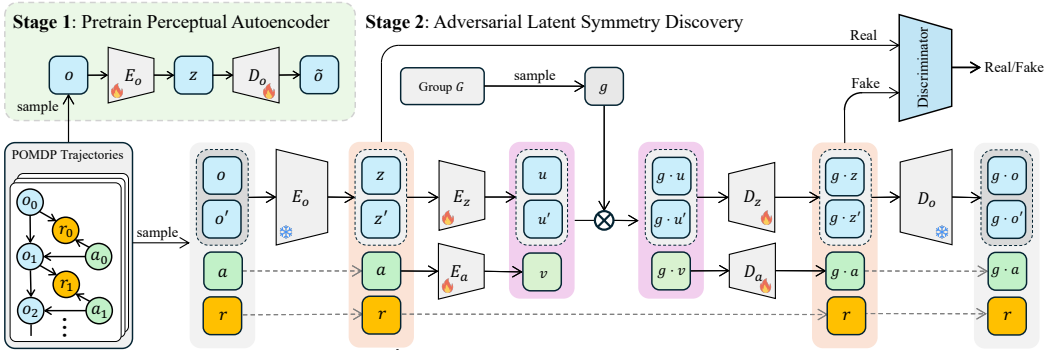


Figure 2: **Two-stage latent symmetry discovery.** Stage 1 trains a perceptual autoencoder (E_o, D_o) to reconstruct observations and then freezes it. In Stage 2, separate symmetry modules encode the perceptual latents (z, z') and action a into symmetry spaces, apply fixed linear representations of the same sampled $g \in G$, and decode the transformed outputs. Adversarial training uses a tuple discriminator to match original and transformed latent transition-tuple distributions, with rewards unchanged. Group-consistency regularization penalizes violations of identity, compatibility, and invertibility.

Given a known symmetry group G and an offline dataset $\mathcal{D} = \{(o, a, o', r)\}$ from a POMDP, our goal is to learn coupled observation–action transformations (ϕ_o, ϕ_a) that preserve the transition-tuple distribution and satisfy the group-operation axioms. Joint tuple matching encourages the transformed observations and actions to remain consistent with the recorded transitions and rewards. Figure 2 summarizes MIRROR; Section 4.1 describes the latent parameterization, and Section 4.2 presents the learning objective.

4.1 Latent group-operation parameterization

Stage 1: Perceptual autoencoder. To reduce the cost of symmetry discovery from pixel observations, we pretrain a perceptual autoencoder (E_o, D_o) on $\mathcal{D}$ for reconstruction and then freeze it (Appendix C). The encoder provides a compact perceptual latent $z = E_o(o) \in \mathbb{R}^{d_z}$ on which the subsequent symmetry modules operate [19, 20, 21, 22]. Appendix A.1 gives the conditions under which latent group operations preserve the group-operation axioms in observation space.

Stage 2: Encode–transform–decode template for group operations. Following Yang et al. [11], we parameterize observation and action group operations using separate encode–transform–decode modules. The observation symmetry module (E_z, D_z) encodes the Stage 1 perceptual latent $z = E_o(o)$ into an observation symmetry latent $u = E_z(z) \in \mathbb{R}^{d_u}$, while the action symmetry module (E_a, D_a) encodes a into an action symmetry latent $v = E_a(a) \in \mathbb{R}^{d_v}$. The fixed linear representations $\rho_z(g)$ and $\rho_a(g)$ act on u and v, respectively. Decoding the transformed codes gives $\phi_z(g, z) := D_z(\rho_z(g)u)$ and $\phi_a(g, a) := D_a(\rho_a(g)v)$, returning to the perceptual latent space and action space, respectively. The frozen perceptual decoder then induces the observation-space operation $\phi_o(g, o) := D_o(\phi_z(g, E_o(o)))$.

4.2 Learning latent symmetry from offline transition tuples

Adversarial tuple matching in latent space. We learn coupled observation–action transformations by matching the joint distribution of transition tuples. This encourages the transformed observations and actions to remain consistent with the recorded transitions and rewards. For each

transition tuple $\tau = (o, a, o', r)$, we form the latent transition tuple $t = (z, a, z', r)$, where $z = E_o(o)$ and $z' = E_o(o')$. Using the same sampled $g \in G$, we transform both z and z' with ϕ_z and the action with ϕ_a, leaving the reward unchanged. This defines the latent tuple map $\Phi_z(g, t) := (\phi_z(g, z), \phi_a(g, a), \phi_z(g, z'), r)$. A discriminator f_ψ learns to distinguish original and transformed tuples, while the symmetry modules $\Theta = (E_z, D_z, E_a, D_a)$ learn to match their distributions through the following adversarial tuple-matching objective:

$$\mathcal{L}_{\text{adv}}(\Theta, \psi) := \mathbb{E}_t\big[f_\psi(t)\big] - \mathbb{E}_{t,\,g}\big[f_\psi(\Phi_z(g, t))\big], \tag{1}$$

optimized as $\min_\Theta \max_\psi \mathcal{L}_{\text{adv}}$. Implementation details are provided in Appendix C.

Group-consistency regularization. Although the fixed linear representations satisfy the representation identities exactly, learned encoding and decoding can introduce consistency errors. We therefore use three regularizers. *Identity* encourages the identity element to leave a tuple unchanged. *Compatibility* encourages successive transformations to agree with their composition. *Invertibility* encourages a transformation followed by its inverse to recover the original tuple. The corresponding losses are

$$\begin{aligned} \mathcal{L}_{\text{id}} &= \mathbb{E}_t\Big[\|t - \Phi_z(e, t)\|_2^2\Big], \\ \mathcal{L}_{\text{comp}} &= \mathbb{E}_{t,g_1,g_2}\Big[\|\Phi_z(g_2, \Phi_z(g_1, t)) - \Phi_z(g_2 \circ g_1, t)\|_2^2\Big], \\ \mathcal{L}_{\text{inv}} &= \mathbb{E}_{t,g}\Big[\big\|\Phi_z(g^{-1}, \Phi_z(g, t)) - t\big\|_2^2\Big]. \end{aligned} \tag{2}$$

Discouraging trivial solutions. Distribution matching and group consistency also admit a trivial solution: every group element leaves all observations and actions unchanged. Following LaLiGAN [11], we discourage this solution with a nontriviality penalty $\mathcal{L}_{\text{nt}}$ based on the cosine similarity between original and transformed perceptual latents.

Overall objective. We train the symmetry modules to match original and transformed latent transition-tuple distributions, satisfy the group-operation axioms, and avoid trivial identity transformations. These goals give the weighted objective:

$$\mathcal{L}_{\text{total}}(\Theta; \psi) := \underbrace{\lambda_{\text{adv}}\mathcal{L}_{\text{adv}}}_{\text{adversarial tuple matching}} + \underbrace{\lambda_{\text{id}}\mathcal{L}_{\text{id}} + \lambda_{\text{comp}}\mathcal{L}_{\text{comp}} + \lambda_{\text{inv}}\mathcal{L}_{\text{inv}}}_{\text{group consistency}} + \underbrace{\lambda_{\text{nt}}\mathcal{L}_{\text{nt}}}_{\text{nontriviality}}. \tag{3}$$

We alternate discriminator and symmetry-module updates (Appendix C). After training, we obtain the coupled observation–action transformations ϕ_o and ϕ_a.

5 Experiments

We first visualize the learned transformations, then compare symmetry quality against baselines and ablations, and finally evaluate downstream policy learning. We study planar rotational symmetry ($G = SO(2)$) on four custom Robosuite [12] tasks: **Drawer Opening**, **Drawer Closing**, **Cube Dropping**, and **Hammer Lifting** (Figure 3).

For each task, we collect 100,000 training and 10,000 evaluation transition tuples using a finite-state-machine controller. Observations are RGB images from a fixed camera, and actions are end-effector commands normalized to $[-1, 1]$. Action coordinates, rewards, and data collection details are given in Appendix B.

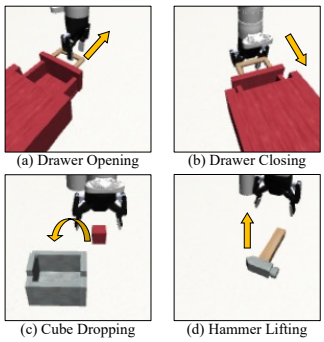


Figure 3: **Simulated manipulation tasks.** The four Robosuite tasks use RGB observations from a fixed camera. Arrows indicate the intended object motion.

5.1 What does MIRROR discover?

MIRROR learns how to change a scene and its associated robot command together. In Figure 4, the drawer's appearance changes

jointly with the planar motion command $(\Delta x, \Delta y)$, while vertical translation Δz and yaw $\Delta\psi$ remain approximately unchanged. This illustrates the relationship motivated in the introduction: changing the task configuration also requires changing the robot's motion.

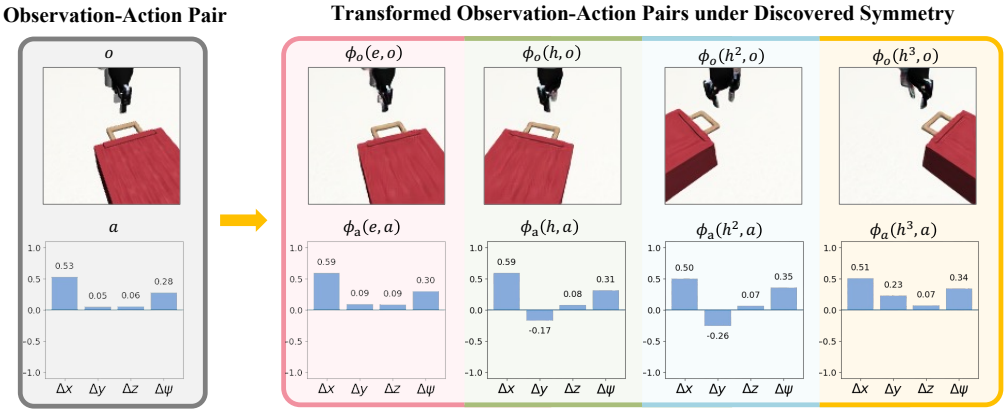


Figure 4: **MIRROR learns coupled changes to the scene and robot command.** In this Drawer Opening example, the original pair (o, a) is shown on the left, followed by four learned pairs $(\phi_o(g, o), \phi_a(g, a))$ for $g \in C_4 = \{e, h, h^2, h^3\}$. The images show the scene configurations, and the bars show the corresponding normalized commands on a $[-1, 1]$ scale. The columns use latent rotation angles $0°, 90°, 180°, 270°$; the decoded transformations need not be geometric rotations by these angles.

The image and command transformations are learned from offline interaction tuples for a specified group. Similar coupling appears across all four tasks (Appendix D.3). We next evaluate whether these transformations preserve the data distribution and satisfy the group-operation axioms.

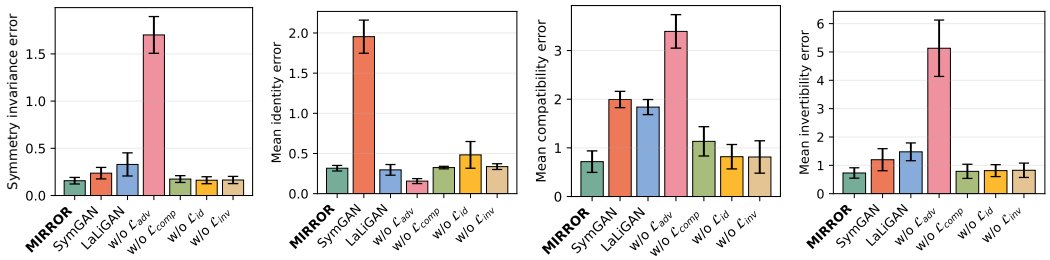
5.2 How accurately does MIRROR recover observation–action symmetries?

Symmetry-quality metrics. We evaluate the learned transformations with four complementary checks. **Symmetry invariance error** measures how closely transformed transition tuples match the original data distribution. **Identity error** checks that applying the identity leaves the input unchanged. **Compatibility error** measures composition consistency: applying two transformations successively should agree with applying their combination in one step. **Invertibility error** checks that a transformation followed by its inverse recovers the input. Lower is better for all four metrics; full definitions are in Appendix B.5.

Baselines and ablations. We adapt two GAN-based baselines to the same tuple formulation and architecture. Within this shared setting, **SymGAN** uses adversarial tuple matching [17, 16, 18], while **LaLiGAN** additionally regularizes identity consistency [11]. Neither adapted baseline includes compatibility or invertibility regularization. To assess the contribution of each loss term, we remove the adversarial tuple-matching, identity, compatibility, or invertibility term from MIRROR one at a time. We use three training seeds per task and the evaluation angles specified in Appendix B.5. Figure 5 summarizes all four tasks; per-task results are in Appendix D.

Distribution preservation. MIRROR achieves the lowest aggregate symmetry invariance error (Figure 5(a); Table 1), supported qualitatively by increasing overlap of original and transformed latent data during training (Appendix D.3). Removing adversarial tuple matching raises this error from 0.157 to 1.701: group-consistency regularization alone does not preserve the interaction-data distribution in our experiments. This ablation also produces the largest aggregate compatibility and invertibility errors, despite lower identity error.

Group consistency. Both GAN baselines exhibit substantially higher composition errors: aggregate compatibility error is 1.994 for SymGAN and 1.838 for LaLiGAN, versus 0.717 for MIRROR—a reduction of at least 61% (Figure 5(c); Table 1). LaLiGAN's slightly lower identity error does not translate into consistent composition: reconstructing the input under the identity



(a) Symmetry invariance (b) Identity (c) Compatibility (d) Invertibility

Figure 5: **Symmetry quality across four manipulation tasks.** Bars show mean errors and error bars indicate ± 1 standard deviation across twelve task–seed results (four tasks, three seeds each), after averaging angle-dependent errors within each run. The "w/o" variants remove one loss term from MIRROR. Lower is better for all metrics. MIRROR has the lowest aggregate symmetry invariance, compatibility, and invertibility errors.

Table 1: **Symmetry-quality comparison and loss ablations** across four tasks (mean $\pm$ standard deviation over twelve task–seed results; lower is better). Rows below the divider remove one loss term from Eq. (3) relative to MIRROR. Bold indicates the lowest error in each column across all rows.

Method	Symmetry invariance	Identity	Compatibility	Invertibility
MIRROR (ours)	$\mathbf{0.157 \pm 0.035}$	0.317 ± 0.034	$\mathbf{0.717 \pm 0.221}$	$\mathbf{0.730 \pm 0.183}$
SymGAN	0.237 ± 0.061	1.954 ± 0.206	1.994 ± 0.167	1.198 ± 0.389
LaLiGAN	0.329 ± 0.122	0.296 ± 0.066	1.838 ± 0.155	1.476 ± 0.314
w/o $\mathcal{L}_{\text{adv}}$	1.701 ± 0.195	$\mathbf{0.156 \pm 0.031}$	3.394 ± 0.344	5.131 ± 0.995
w/o $\mathcal{L}_{\text{id}}$	0.162 ± 0.037	0.483 ± 0.166	0.818 ± 0.251	0.814 ± 0.211
w/o $\mathcal{L}_{\text{comp}}$	0.174 ± 0.036	0.325 ± 0.016	1.134 ± 0.301	0.788 ± 0.250
w/o $\mathcal{L}_{\text{inv}}$	0.164 ± 0.038	0.337 ± 0.036	0.813 ± 0.333	0.826 ± 0.253

is insufficient to ensure that different transformations work together. The full model also reduces aggregate invertibility error by at least 39% relative to either GAN baseline (Figure 5(d)).

The loss ablations reveal an asymmetry among the group-consistency regularizers. Removing $\mathcal{L}_{\text{comp}}$, while retaining identity and invertibility regularization, increases aggregate compatibility error from 0.717 to 1.134. Removing identity or invertibility regularization produces smaller increases, to 0.818 and 0.813, respectively (Table 1).

Why explicitly regularize composition? This asymmetry is consistent with the logical dependence among the underlying axioms. Appendix A.2 proves that, when satisfied exactly, composition consistency together with either identity or invertibility implies the remaining axiom. In contrast, identity and invertibility together do not imply composition consistency.

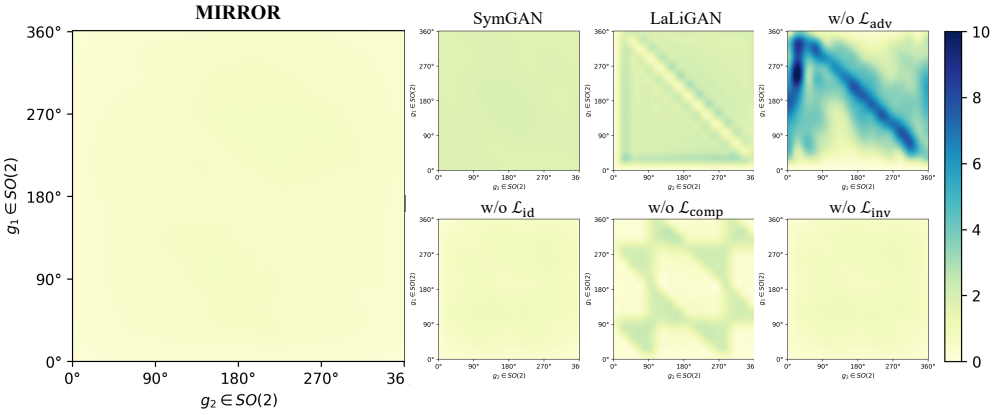


Figure 6: **Compatibility error heatmaps on Drawer Opening.** Errors compare successive transformations with their composition at all 256 ordered pairs of 16 equally spaced angles; heatmaps are smoothed for visualization. MIRROR (large left panel), baselines, and single-loss ablations share a color scale (lower is better). MIRROR maintains low error across the evaluated pairs.

Composition therefore supplies a constraint on how different transformations combine that reconstruction and inverse checks alone cannot ensure. Figure 6 illustrates this distinction on Drawer Opening. MIRROR maintains low compatibility error across the evaluated angle pairs, while removing $\mathcal{L}_{\text{comp}}$ introduces regions of elevated error despite retaining identity and invertibility regularization. Removing $\mathcal{L}_{\text{id}}$ or $\mathcal{L}_{\text{inv}}$ produces smaller changes across the grid. These results support explicitly regularizing composition. The theoretical result concerns exact constraints, so it does not imply that identity and invertibility penalties are interchangeable during training.

5.3 Do the discovered symmetries improve robot policy learning?

Having evaluated symmetry quality, we now test whether the learned transformations benefit policy learning. We use frame-averaged BC [23]: predict actions from transformed observations, map them back to the original action frame, and average them. Experimental method names use the prefix EBC (equivariant behavior cloning), giving EBC-MIRROR for our transformations. The policy averages over n transformations from the cyclic subgroup $H = C_n \subset SO(2)$; construction, training details, and conditions for exact equivariance are given in Appendix E.

EBC-MIRROR improves mean success on all four tasks, raising the average from 80.5% for vanilla BC to 85.1% at $H = C_{16}$ (Table 2). The largest gain is on **Cube Dropping** ($+9.7$ percentage points). Frame-averaged BC policies using hand-designed (EBC-ExSym [24]) or GAN-discovered (EBC-SymGAN and EBC-LaLiGAN) symmetries all have lower average success than vanilla BC.

Table 2: Episodic success rates in percent (mean $\pm$ std) at $H = C_{16}$, with vanilla BC as reference. **Bold** indicates the best per column. Results for smaller subgroups (C_2, C_4, C_8) are in Appendix E.4.

Method	Drawer Opening	Drawer Closing	Cube Dropping	Hammer Lifting	Average
EBC-ExSym (C_{16})	64.7 ± 2.2	86.4 ± 4.7	80.9 ± 5.9	85.1 ± 5.7	79.3 ± 9.9
EBC-SymGAN (C_{16})	66.3 ± 11.3	85.8 ± 3.6	81.9 ± 9.4	49.8 ± 16.3	71.0 ± 18.1
EBC-LaLiGAN (C_{16})	65.3 ± 2.0	73.6 ± 6.2	84.5 ± 6.2	61.3 ± 5.6	71.2 ± 10.3
EBC-MIRROR (C_{16})	$\mathbf{71.7 \pm 1.4}$	$\mathbf{89.8 \pm 2.0}$	$\mathbf{90.3 \pm 2.9}$	$\mathbf{88.7 \pm 2.5}$	$\mathbf{85.1 \pm 8.1}$
BC	71.4 ± 0.8	88.1 ± 5.9	80.6 ± 2.5	81.8 ± 3.0	80.5 ± 7.0

Effect of subgroup size. Averaging over more transformations steadily improves EBC-MIRROR's mean success rate across the four tasks, whereas EBC-SymGAN and EBC-LaLiGAN decline overall (Figure 7). EBC-ExSym remains below vanilla BC throughout the sweep. These trends show that the benefit of frame averaging depends on the quality of the underlying symmetries.

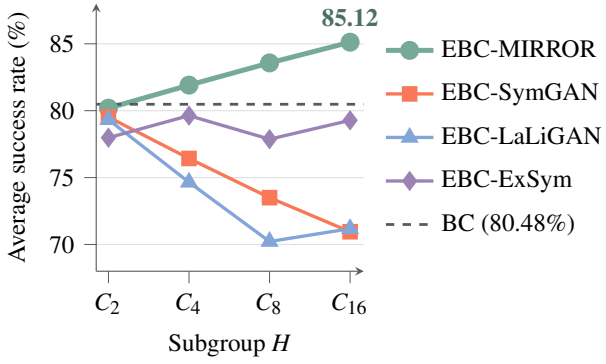


Figure 7: **RGB policy performance versus subgroup size.** Points show mean episodic success rates across four tasks; higher is better. For $H = C_n$, the policy uses n symmetry transformations for frame averaging. The dashed line denotes vanilla BC. EBC-MIRROR improves with subgroup size and exceeds BC at C_4, C_8, and C_{16}; the other frame-averaged variants remain below BC. Per-task results and standard deviations are reported in Tables 2 and 10.

6 Sim-to-Real Deployment

To test transfer beyond RGB, we use two-channel observations comprising a depth-derived heightmap and a target-object segmentation mask (Figure 8(e,f)). We rerun symmetry discovery in simulation, freeze the perceptual autoencoder and symmetry modules, and train EBC-MIRROR on this modality (Appendix F.3) before *zero-shot* deployment on a Kinova GEN3 arm. Appendix F.5 visualizes the coupled observation–action transformations.

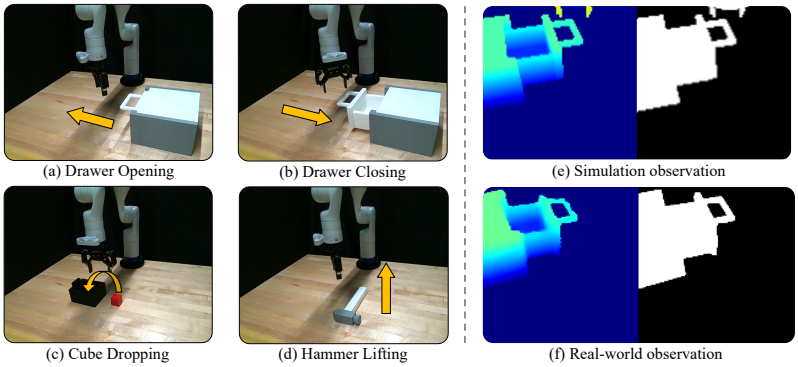


Figure 8: **Real-world tasks and policy observations.** (a)–(d) Arrows indicate intended object motion. (e,f) Simulation and real-world observations: heightmap (left) and segmentation mask (right).

Over 20 trials per task, the policy succeeds on 13/20 Drawer Opening, 19/20 Drawer Closing, 9/20 Cube Dropping, and 16/20 Hammer Lifting, transferring on three of four tasks with Cube Dropping remaining the most challenging. Hardware and preprocessing details are in Appendix F.1.

7 Conclusion

We introduced **MIRROR**, a framework for learning how a specified symmetry group operates jointly on observations and actions in pixel-based POMDPs from offline interaction data. We formulate observable symmetry as invariance of the offline one-step transition-tuple distribution. Adversarial tuple matching learns coupled transformations, while identity, compatibility, and invertibility regularization encourages consistency with the group-operation axioms. Across four Robosuite manipulation tasks, MIRROR achieves the lowest aggregate error on three of four symmetry-quality metrics, including a 61% reduction in compatibility error relative to the adapted LaLiGAN baseline.

Using these transformations in frame-averaged BC yields higher mean success rates on all four tasks than vanilla BC and the hand-designed and GAN-discovered symmetry baselines at $H = C_{16}$. Gains over vanilla BC reach 9.66 percentage points on Cube Dropping and 4.64 percentage points on average. We also repeat symmetry discovery and policy training in simulation with heightmap and segmentation-mask observations, then deploy the resulting policy zero-shot on a Kinova GEN3 arm. Together, these results show that learning coupled observation–action transformations from data can improve visual policy learning without manually specifying how a known symmetry group operates on images and robot commands.

8 Limitations

MIRROR has two scope boundaries worth naming. First, the framework discovers how a group operates on observations and actions but requires the group G itself to be specified in advance; we view this as a mild assumption in practice, since the relevant group is usually evident from task geometry while its observation–action transformations are what is genuinely hard to specify by hand. Second, MIRROR can only recover symmetries that are already present in the data: if the dataset lacks variation along the symmetry (for example, a fixed scene orientation across all episodes), the discriminator has nothing to match against, and (ϕ_o, ϕ_a) may collapse toward the identity.

Acknowledgments

We thank Haohong Lin and William Jongwon Han for their constructive feedback and informative discussions.

References

[1] D. Wang, R. Walters, and R. Platt. $\mathrm{SO}(2)$-equivariant reinforcement learning. In *International Conference on Learning Representations*, 2022. URL https://openreview.net/forum?id=7F9cOhdvfk_.

[2] S. Liu, M. Xu, P. Huang, X. Zhang, Y. Liu, K. Oguchi, and D. Zhao. Continual vision-based reinforcement learning with group symmetries. In *Conference on Robot Learning*, pages 222–240. PMLR, 2023.

[3] H. Ryu, H. in Lee, J.-H. Lee, and J. Choi. Equivariant descriptor fields: SE(3)-equivariant energy-based models for end-to-end visual robotic manipulation learning. In *The Eleventh International Conference on Learning Representations*, 2023. URL https://openreview.net/forum?id=dnjZSPGmY5O.

[4] H. Huang, O. L. Howell, D. Wang, X. Zhu, R. Platt, and R. Walters. Fourier transporter: Bi-equivariant robotic manipulation in 3d. In *The Twelfth International Conference on Learning Representations*, 2024. URL https://openreview.net/forum?id=UulwvAU1W0.

[5] B. Eisner, Y. Yang, T. Davchev, M. Vecerik, J. Scholz, and D. Held. Deep SE(3)-equivariant geometric reasoning for precise placement tasks. In *The Twelfth International Conference on Learning Representations*, 2024. URL https://openreview.net/forum?id=2inBuwTyL2.

[6] J. Yang, Z.-a. Cao, C. Deng, R. Antonova, S. Song, and J. Bohg. Equibot: Sim(3)-equivariant diffusion policy for generalizable and data efficient learning. In *8th Annual Conference on Robot Learning*, 2024.

[7] D. Wang, S. Hart, D. Surovik, T. Kelestemur, H. Huang, H. Zhao, M. Yeatman, J. Wang, R. Walters, and R. Platt. Equivariant diffusion policy. In *8th Annual Conference on Robot Learning*, 2024. URL https://openreview.net/forum?id=wD2kUVLT1g.

[8] C. Gao, Z. Xue, S. Deng, T. Liang, S. Yang, L. Shao, and H. Xu. RiEMann: Near real-time SE(3)-equivariant robot manipulation without point cloud segmentation. In *8th Annual Conference on Robot Learning*, 2024. URL https://openreview.net/forum?id=eJHy0AF5TO.

[9] B. Hu, D. Wang, D. Klee, H. Tian, X. Zhu, H. Huang, R. Platt, and R. Walters. 3d equivariant visuomotor policy learning via spherical projection. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025. URL https://openreview.net/forum?id=kXJd4JxF34.

[10] D. Klee, B. Hu, A. Cole, H. Tian, D. Wang, R. Platt, and R. Walters. RAVEN: End-to-end equivariant robot learning with RGB cameras. In *The Fourteenth International Conference on Learning Representations*, 2026. URL https://openreview.net/forum?id=z8BN7KyaPl.

[11] J. Yang, N. Dehmamy, R. Walters, and R. Yu. Latent space symmetry discovery. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, volume 235 of *Proceedings of Machine Learning Research*. PMLR, 2024. URL https://proceedings.mlr.press/v235/yang24g.html.

[12] Y. Zhu, J. Wong, A. Mandlekar, R. Martín-Martín, A. Joshi, K. Lin, S. Nasiriany, and Y. Zhu. robosuite: A modular simulation framework and benchmark for robot learning. In *arXiv preprint arXiv:2009.12293*, 2020.

[13] N. Dehmamy, R. Walters, Y. Liu, D. Wang, and R. Yu. Automatic symmetry discovery with lie algebra convolutional network. *Advances in Neural Information Processing Systems*, 34: 2503–2515, 2021.

[14] A. Moskalev, A. Sepliarskaia, I. Sosnovik, and A. Smeulders. Liegg: Studying learned lie group generators. *Advances in Neural Information Processing Systems*, 35:25212–25223, 2022.

[15] L. Hu, Y. Li, and Z. Lin. Explicit discovery of nonlinear symmetries from dynamic data. In *Forty-second International Conference on Machine Learning*, 2025. URL https://openreview.net/forum?id=kzY6nbbBYq.

[16] J. Yang, R. Walters, N. Dehmamy, and R. Yu. Generative adversarial symmetry discovery. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, volume 202 of *Proceedings of Machine Learning Research*. PMLR, 2023. URL https://proceedings.mlr.press/v202/yang23n.html.

[17] K. Desai, B. Nachman, and J. Thaler. Symmetry discovery with deep learning. *Physical Review D*, 105(9):096031, 2022.

[18] J. Liu, Y. Yang, V. Bansal, and C. Allen-Blanchette. Physically plausible multi-system trajectory generation and symmetry discovery. *arXiv preprint arXiv:2509.23003*, 2025.

[19] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer. High-resolution image synthesis with latent diffusion models. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 10684–10695, 2022.

[20] A. Van Den Oord, O. Vinyals, et al. Neural discrete representation learning. *Advances in neural information processing systems*, 30, 2017.

[21] A. Razavi, A. Van den Oord, and O. Vinyals. Generating diverse high-fidelity images with vq-vae-2. *Advances in neural information processing systems*, 32, 2019.

[22] P. Esser, R. Rombach, and B. Ommer. Taming transformers for high-resolution image synthesis. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 12873–12883, 2021.

[23] O. Puny, M. Atzmon, E. J. Smith, I. Misra, A. Grover, H. Ben-Hamu, and Y. Lipman. Frame averaging for invariant and equivariant network design. In *International Conference on Learning Representations*, 2022. URL https://openreview.net/forum?id=zIUyj55nXR.

[24] D. Wang, J. Y. Park, N. Sortur, L. L. Wong, R. Walters, and R. Platt. The surprising effectiveness of equivariant models in domains with latent symmetry. In *International Conference on Learning Representations*, 2023. URL https://openreview.net/forum?id=P4MUGRM4Acu.

[25] D. Zhao, J. Zhu, and B. Zhang. Latent variables on spheres for autoencoders in high dimensions, 2020. URL https://arxiv.org/abs/1912.10233.

[26] N. Ravi, V. Gabeur, Y.-T. Hu, R. Hu, C. Ryali, T. Ma, H. Khedr, R. Rädle, C. Rolland, L. Gustafson, E. Mintun, J. Pan, K. V. Alwala, N. Carion, C.-Y. Wu, R. Girshick, P. Dollar, and C. Feichtenhofer. SAM 2: Segment anything in images and videos. In *The Thirteenth International Conference on Learning Representations*, 2025. URL https://openreview.net/forum?id=Ha6RTeWMd0.

[27] K. Lin, V. Ragunath, A. McAlinden, A. Prasad, J. Wu, Y. Zhu, and J. Bohg. Constraint-preserving data generation for one-shot visuomotor policy generalization. In *9th Annual Conference on Robot Learning*, 2025. URL https://openreview.net/forum?id=KSKzA1mwKs.

Appendix Overview

- **Appendix A: Theoretical Analysis**
- **Appendix B: Simulation Setup and Evaluation Protocol**
- **Appendix C: Symmetry-Discovery Implementation**
- **Appendix D: Additional Symmetry-Discovery Results**
- **Appendix E: Frame-Averaged Behavior Cloning**
- **Appendix F: Sim-to-Real Experiments**

A Theoretical Analysis

A.1 Preservation of Group-Operation Axioms under Two-Stage Encoding

We use a two-stage observation encoding scheme: a fixed perceptual autoencoder (E_o, D_o) maps observations to a compact perceptual latent space, and the learnable symmetry modules operate on this latent space. This appendix establishes exact preservation of the group-operation axioms when the latent operation satisfies them and the perceptual encoder and decoder are mutual inverses on the relevant invariant support.

Setup. Let G be a group with identity element e and composition $\circ$. Let $E_o : \mathcal{O} \to \mathcal{Z}$ and $D_o : \mathcal{Z} \to \mathcal{O}$ be the fixed perceptual encoder/decoder, and let $\phi_z : G \times \mathcal{Z} \to \mathcal{Z}$ be the learned transformation on perceptual latents. We define the induced observation-space transformation by

$$\phi_o(g, o) \;:=\; D_o\big(\phi_z(g, E_o(o))\big). \tag{4}$$

Lemma 1 (Conjugation preserves group operations)**.** *Let $\mathcal{O}_* \subseteq \mathcal{O}$ be a relevant observation support and define $\mathcal{Z}_* = E_o(\mathcal{O}_*)$. Assume that $\mathcal{Z}_*$ is G-invariant under ϕ_z, meaning $\phi_z(g, z) \in \mathcal{Z}_*$ for every $g \in G$ and $z \in \mathcal{Z}_*$. Assume that ϕ_z is a (left) group operation on $\mathcal{Z}_*$, i.e., for every $z \in \mathcal{Z}_*$,*

$$\phi_z(e, z) = z, \tag{5}$$

$$\phi_z(g_2, \phi_z(g_1, z)) = \phi_z(g_2 \circ g_1, z), \quad \forall g_1, g_2 \in G. \tag{6}$$

Further assume that (E_o, D_o) are mutual inverses on these supports, meaning

$$D_o(E_o(o)) = o \quad (o \in \mathcal{O}_*), \qquad E_o(D_o(z)) = z \quad (z \in \mathcal{Z}_*). \tag{7}$$

Then the induced map ϕ_o in (4) is a group operation on $\mathcal{O}_*$: for every $o \in \mathcal{O}_*$,

$$\phi_o(e, o) = o, \tag{8}$$

$$\phi_o(g_2, \phi_o(g_1, o)) = \phi_o(g_2 \circ g_1, o), \quad \forall g_1, g_2 \in G. \tag{9}$$

Proof. Fix $o \in \mathcal{O}_*$. Since $D_o(\mathcal{Z}_*) = \mathcal{O}_*$ and $\mathcal{Z}_*$ is G-invariant, $\phi_o(g, o) \in \mathcal{O}_*$ for every $g \in G$, so the induced operation is closed on the stated support.

Identity. Using (4), (5), and (7),

$$\phi_o(e, o) = D_o\big(\phi_z(e, E_o(o))\big) = D_o\big(E_o(o)\big) = o.$$

Compatibility. Using (4) twice, then (7) to cancel $E_o \circ D_o$, and finally (6),

$$\begin{aligned}
\phi_o(g_2, \phi_o(g_1, o)) &= D_o\Big(\phi_z\big(g_2,\ E_o(\phi_o(g_1, o))\big)\Big) \\
&= D_o\Big(\phi_z\big(g_2,\ E_o(D_o(\phi_z(g_1, E_o(o))))\big)\Big) \\
&= D_o\Big(\phi_z\big(g_2,\ \phi_z(g_1, E_o(o))\big)\Big) \\
&= D_o\Big(\phi_z\big(g_2 \circ g_1,\ E_o(o)\big)\Big) \\
&= \phi_o(g_2 \circ g_1, o).
\end{aligned}$$

This proves (8)–(9). Invertibility follows by taking $g_2 = g_1^{-1}$ in the compatibility identity and applying the identity axiom. □

Interpretation. Under the lemma's assumptions, the perceptual autoencoder is a change of coordinates between $\mathcal{O}_*$ and $\mathcal{Z}_*$. An exact latent group operation therefore induces an exact observation-space group operation on the corresponding support. The conclusion depends on the stated inverse and invariance conditions; pretraining and freezing the autoencoder alone does not guarantee them.

Remark (learned approximations). The lemma establishes exact preservation under its stated assumptions. In practice, the perceptual autoencoder is approximately invertible, and the learned latent operation is encouraged to satisfy the group-operation axioms through regularization. Only the fixed linear representations ρ_z and ρ_a satisfy the representation identities exactly by construction. Consequently, the learned observation-space operation may exhibit residual axiom violations, which we quantify using the evaluation metrics in Appendix B.5.

A.2 Logical Dependence among the Group-Operation Axioms

The MIRROR objective uses three group-consistency regularizers: identity ($\mathcal{L}_{\text{id}}$), compatibility ($\mathcal{L}_{\text{comp}}$), and invertibility ($\mathcal{L}_{\text{inv}}$) in Eq. (2). This section analyzes the logical dependence among the corresponding exact constraints: given compatibility, identity and invertibility imply one another, whereas identity and invertibility together do not imply compatibility.

Setup. Let G be a group with identity element e and composition $\circ$, and let $\phi : G \times \mathcal{X} \to \mathcal{X}$ denote a candidate group operation on a set $\mathcal{X}$. The analysis applies to both Φ and Φ_z on their respective tuple spaces. We consider the following axioms, each required to hold exactly for every $g, g_1, g_2 \in G$ and $x \in \mathcal{X}$:

$$\text{(Identity)} \qquad \phi(e, x) = x, \tag{10}$$

$$\text{(Compatibility)} \qquad \phi(g_2, \phi(g_1, x)) = \phi(g_2 \circ g_1, x), \tag{11}$$

$$\text{(Invertibility)} \qquad \phi(g^{-1}, \phi(g, x)) = x. \tag{12}$$

Lemma 2 (Identity from compatibility and invertibility)**.** *If ϕ satisfies compatibility* (11) *and invertibility* (12)*, then ϕ satisfies identity* (10)*.*

Proof. Fix any $g \in G$ and any $x \in \mathcal{X}$. Since $e = g^{-1} \circ g$,

$$\phi(e, x) \;=\; \phi(g^{-1} \circ g, x) \overset{(11)}{=} \phi(g^{-1}, \phi(g, x)) \overset{(12)}{=} x,$$

which is the identity axiom. □

Lemma 3 (Invertibility from compatibility and identity)**.** *If ϕ satisfies compatibility* (11) *and identity* (10)*, then ϕ satisfies invertibility* (12)*.*

Proof. For any $g \in G$ and $x \in \mathcal{X}$,

$$\phi(g^{-1}, \phi(g, x)) \overset{(11)}{=} \phi(g^{-1} \circ g, x) \;=\; \phi(e, x) \overset{(10)}{=} x,$$

which is the invertibility axiom. □

Independence of compatibility. Identity and invertibility together do not imply compatibility. To see this, take $G = SO(2)$ and $\mathcal{X} = \mathbb{R}$, and define $\phi(R_\theta, x) = x + \sin\theta$, where R_θ denotes rotation by angle θ. This map is well-defined on $SO(2)$ because $\sin\theta$ is 2π-periodic, and it satisfies identity and invertibility for every $x \in \mathbb{R}$ and θ:

$$\phi(R_0, x) = x, \qquad \phi(R_{-\theta}, \phi(R_\theta, x)) = x + \sin\theta - \sin\theta = x.$$

However, compatibility fails, since

$$\phi(R_{\pi/2}, \phi(R_{\pi/2}, x)) = x + 2 \neq x = \phi(R_\pi, x).$$

Consequence for the exact constraints. Lemmas 2 and 3 establish that, under exact compatibility, identity and invertibility imply one another. Consequently, either identity or invertibility is redundant in an exact characterization that includes the other two axioms, while compatibility cannot be inferred from identity and invertibility alone. This logical dependence concerns the axiom constraints themselves and does not imply that their corresponding optimization penalties are interchangeable.

B Simulation Setup and Evaluation Protocol

We first specify the simulation setup, observation and action coordinates, and per-task reward functions. We then describe offline data collection in Section B.4, followed by the evaluation metrics and group sampling in Section B.5.

B.1 Robosuite Setup

All four tabletop tasks share the same Robosuite hardware and perception configuration, chosen to mirror our physical platform so that the same observation and action interfaces can be reused in the sim-to-real study of Appendix F.1. The simulated arm is a 7-DoF Kinova GEN3 manipulator equipped with a Robotiq 2F85 parallel-jaw gripper, matching the real robot used for zero-shot transfer. Rather than adopting one of Robosuite's built-in camera presets, we calibrate the simulator camera against the real workspace: we first mount an Intel RealSense D435 in front of the table and run a standard eye-to-hand calibration to recover its intrinsic matrix and its extrinsic pose with respect to the robot base, and we then transcribe both matrices into the simulator so that the virtual camera reproduces the same viewpoint, field of view, and projection geometry as the physical sensor. This calibration-driven setup yields RGB observations whose camera geometry is aligned between simulation and the real robot, eliminating viewpoint-induced sim-to-real gap before any policy transfer.

B.2 Per-Task Observation and Action Coordinates

The observation space $\mathcal{O}$ is the same 3-channel RGB image from a fixed camera (Figure 3) for all four tasks; the action space $\mathcal{A}$ consists of task-dependent, normalized end-effector commands. Throughout the experiments and action visualizations, $(\Delta x, \Delta y, \Delta z)$, $\Delta\psi$, and Δw denote dimensionless, normalized commands for Cartesian translation, yaw, and gripper opening, respectively. For Drawer Opening and Drawer Closing, actions are 4-dimensional, $(\Delta x, \Delta y, \Delta z, \Delta\psi)$. For Cube Dropping, actions are also 4-dimensional, $(\Delta x, \Delta y, \Delta z, \Delta w)$. For Hammer Lifting, actions are 5-dimensional, $(\Delta x, \Delta y, \Delta z, \Delta\psi, \Delta w)$. Each action component uses zero-centered scaling, $a_i = \delta_i / s_i$, where δ_i is the physical command increment and $s_i > 0$ is its per-coordinate scale. The normalized commands lie in $[-1, 1]$, so $\mathcal{A} \subseteq [-1, 1]^{d_a}$ with the task-specific dimension d_a given above. The corresponding per-task vector-valued reward functions used by the discriminator are defined in Section B.3 below.

B.3 Environment Reward Functions

The discriminator receives a vector-valued reward whose ordered components are listed in Table 3. Cube Dropping uses six components; the other tasks use four. We define the shared components once below and then specify the task-specific terms.

Shared reaching and action terms. Let $\mathbf{p}_t^{\mathrm{grp}} \in \mathbb{R}^3$ be the gripper position and $\mathbf{p}_t^{\mathrm{target}} \in \mathbb{R}^3$ the position of the task's reaching target (Table 3). All four tasks use

$$
\begin{aligned}
d_t^{\mathrm{reach}} &:= \left\| \mathbf{p}_t^{\mathrm{grp}} - \mathbf{p}_t^{\mathrm{target}} \right\|_2 , \\
r_t^{\mathrm{reach}} &:= 1 - \tanh\!\left(10\, d_t^{\mathrm{reach}}\right) , \qquad r_t^{\mathrm{action}} := - \left\| \mathbf{a}_t \right\|_2 .
\end{aligned}
\tag{13}
$$

Drawer progress and alignment. For both drawer tasks, let $p_t \in [0, 1]$ denote the drawer-out fraction and ϵ_t^{yaw} the yaw difference, in radians, between the gripper and the handle target. The task-specific terms are

$$r_t^{\text{open}} := \tanh(p_t), \qquad r_t^{\text{close}} := 1 - \tanh(p_t), \qquad r_t^{\text{yaw}} := \cos^2(\epsilon_t^{\text{yaw}}). \tag{14}$$

Grasping and lifting. For Cube Dropping and Hammer Lifting, let $g_t \in \{0, 1\}$ indicate whether the respective object is grasped and Δh_t denote its height above the table surface. Both tasks use

$$r_t^{\text{grasp}} := g_t, \qquad r_t^{\text{lift}} := \tanh(10\,\Delta h_t)\,. \tag{15}$$

Cube alignment and success. Let $\mathbf{p}_t^{\text{cube}}, \mathbf{p}_t^{\text{pot}} \in \mathbb{R}^3$ be the cube and pot positions. Alignment above the pot is measured in the horizontal plane:

$$d_t^{\text{align}} := \left\|\mathbf{p}_{t,1:2}^{\text{cube}} - \mathbf{p}_{t,1:2}^{\text{pot}}\right\|_2, \qquad r_t^{\text{align}} := 1 - \tanh\Big(10\, d_t^{\text{align}}\Big)\,. \tag{16}$$

The success reward $r_t^{\text{success}} = 1$ when the cube is in the pot and the gripper has released it.

Table 3: Per-task reward composition. Components are listed in their order in the reward vector; the reaching target specifies $\mathbf{p}_t^{\text{target}}$.

Task	Reaching target	Reward vector $\mathbf{r}_t^\top$
Drawer Opening	Drawer handle	$(r_t^{\text{reach}}, r_t^{\text{open}}, r_t^{\text{yaw}}, r_t^{\text{action}})$
Drawer Closing	Drawer handle	$(r_t^{\text{reach}}, r_t^{\text{close}}, r_t^{\text{yaw}}, r_t^{\text{action}})$
Cube Dropping	Cube	$(r_t^{\text{reach}}, r_t^{\text{grasp}}, r_t^{\text{lift}}, r_t^{\text{align}}, r_t^{\text{success}}, r_t^{\text{action}})$
Hammer Lifting	Hammer	$(r_t^{\text{reach}}, r_t^{\text{grasp}}, r_t^{\text{lift}}, r_t^{\text{action}})$

B.4 Offline Dataset and Behavior Policy

For each task, we collect 100,000 one-step transition tuples (o, a, o', r) for training and 10,000 for evaluation, as described in Section 5. Each tuple contains the current and next RGB observations, the normalized end-effector command, and the task-specific reward vector defined above.

The behavior policy used to collect the offline interaction data for all four tasks is a simple finite-state-machine controller that drives the robot end-effector through a sequence of task-specific waypoints using proportional pose control. The waypoint sequence for each task is listed below.

Drawer Opening.

1. a pre-grasp pose about 5 cm above the drawer handle;
2. a grasp pose about 2 cm below the handle to establish contact;
3. a pulling pose about 10 cm along the drawer-opening direction.

Drawer Closing.

1. an approach pose about 10 cm in front of the handle and 2 cm above it;
2. a contact pose at the same front offset but 2 cm below the handle height;
3. a pushing pose about 10 cm past the handle in the closing direction.

Cube Dropping.

1. an approach pose about 20 cm above the cube with the gripper open;
2. a descend pose about 2 cm below the cube reference position for grasping;
3. a lift pose about 20 cm above the cube with the gripper closed;
4. a transfer pose about 20 cm above the pot;
5. a drop pose at the same hover position above the pot, where the gripper opens to release the cube.

Hammer Lifting.

1. an approach pose about 20 cm above the hammer with the gripper open;
2. a descend pose about 2 cm below the hammer reference position to align for grasping;
3. a grasp pose at the same position with the gripper closed;
4. a lift pose about 20 cm upward.

B.5 Evaluation Metrics and Protocol

We evaluate the learned observation–action transformations (ϕ_o, ϕ_a) along two criteria: (1) *observable symmetry*, i.e., whether transforming tuples preserves the offline transition-tuple distribution; and (2) *group-operation validity*, i.e., whether the learned maps satisfy basic group-operation axioms (identity, compatibility, and invertibility).

B.5.1 Observable Symmetry Evaluation

We measure observable symmetry by the **Symmetry invariance error**, the Wasserstein-2 distance between the original dataset $\mathcal{D}$ and its transformed version $\Phi(g, \mathcal{D})$:

$$\mathcal{E}_{\mathrm{sym}}(g) \;=\; W_2(\mathcal{D}, \Phi(g, \mathcal{D}))\,. \tag{17}$$

A perfect symmetry yields $\mathcal{E}_{\mathrm{sym}}(g) = 0$ (i.e., $p(\tau) = p(\Phi(g, \tau))$), and smaller values mean $\Phi(g, \mathcal{D})$ remains closer in distribution to $\mathcal{D}$.

Ground cost in feature space. Direct optimal transport in raw pixel space is typically uninformative and expensive. Instead, we embed observations using a fixed pretrained visual encoder $f_o : \mathcal{O} \to \mathbb{R}^{d_o}$ (e.g., ResNet-18 features) and compare tuples $\tau = (o, a, o', r)$ and $\tau' = (\bar{o}, \bar{a}, \bar{o}', \bar{r})$ via the per-tuple feature distance

$$d_f(\tau, \tau') \;:=\; \|f_o(o) - f_o(\bar{o})\|_2^2 + \|a - \bar{a}\|_2^2 + \|f_o(o') - f_o(\bar{o}')\|_2^2, \tag{18}$$

adding $\|r - \bar{r}\|_2^2$ as the Wasserstein ground cost (since arbitrary tuples may genuinely differ in reward) and using d_f directly for the validity errors below (since Φ preserves r by construction).

B.5.2 Group-Operation Validity Evaluation

Beyond distribution matching, (ϕ_o, ϕ_a) should also satisfy the three group-operation axioms when applied to tuples: identity should leave a tuple unchanged, applying two transformations in sequence should equal applying their composition, and a transformation followed by its inverse should return to the original. For fixed group elements $g, g_1, g_2 \in G$, we define three feature-space errors

$$\begin{aligned}
\mathcal{E}_{\mathrm{id}} &= \mathbb{E}_{\tau\sim\mathcal{D}}\big[\, d_f(\Phi(e, \tau),\, \tau)\,\big], \\
\mathcal{E}_{\mathrm{comp}}(g_1, g_2) &= \mathbb{E}_{\tau\sim\mathcal{D}}\big[\, d_f(\Phi(g_2, \Phi(g_1, \tau)),\, \Phi(g_2 \circ g_1, \tau))\,\big], \\
\mathcal{E}_{\mathrm{inv}}(g) &= \mathbb{E}_{\tau\sim\mathcal{D}}\big[\, d_f(\Phi(g^{-1}, \Phi(g, \tau)),\, \tau)\,\big].
\end{aligned} \tag{19}$$

B.5.3 Group Sampling for Evaluation

At evaluation time, $\overline{\mathcal{E}}_{\mathrm{sym}}$ and $\overline{\mathcal{E}}_{\mathrm{inv}}$ denote averages of $\mathcal{E}_{\mathrm{sym}}(g)$ and $\mathcal{E}_{\mathrm{inv}}(g)$ over 256 angles uniformly sampled from $SO(2)$; identity error $\mathcal{E}_{\mathrm{id}}$ is evaluated only at $g = e$. For $\overline{\mathcal{E}}_{\mathrm{comp}}$, we average $\mathcal{E}_{\mathrm{comp}}(g_1, g_2)$ over all $16 \times 16 = 256$ ordered pairs of equally spaced rotation angles spanning the full circle. The compatibility heatmaps in Appendix D.2.2 display per-pair errors on this same grid. Qualitative visualizations of the learned transformations and of latent-space distribution matching use the discrete subgroup $C_4 = \{e, h, h^2, h^3\} \subset SO(2)$ with $\theta_h = 90^\circ$ in the linear latent representations.

C Symmetry-Discovery Implementation

This appendix specifies the network architectures (Section C.1), fixed group representations (Section C.2), training objectives and procedure (Section C.3), and hyperparameters (Section C.4) for symmetry discovery. Evaluation metrics and evaluation-time group sampling are described in Appendix B.5, and the frame-averaged BC policy is described in Appendix E.

C.1 Network Architectures

Perceptual autoencoder (E_o, D_o) (Stage 1). We instantiate the perceptual autoencoder as a convolutional spherical autoencoder [25] that maps each workspace observation to a unit-norm perceptual latent and back, providing the smooth perceptual latent space on which Stage 2 symmetry discovery operates. The network accepts a channel-first $C \times 128 \times 128$ tensor with $C = 3$ for the raw RGB images used in the simulation-only experiments and $C = 2$ for the stacked heightmap and segmentation mask used in the sim-to-real pipeline. The encoder is a five-stage convolutional tower with channel widths $4, 8, 16, 32, 64$, SiLU activations, and layer normalization, interleaved with four average-pooling downsamples that produce a $64 \times 8 \times 8$ bottleneck. A linear projection followed by ℓ_2 normalization then sends this bottleneck to a perceptual latent $z = E_o(o) \in \mathbb{S}^{15}$ with ambient dimension $d_z = 16$. The decoder mirrors this structure: a linear expansion lifts z back to a $64 \times 8 \times 8$ feature map, bilinear-upsampling blocks with channel widths $128, 64, 32, 16, 8$ progressively restore spatial resolution, and a final 3×3 convolution followed by a $\tanh$ activation and per-channel range de-normalization recovers the input tensor.

Symmetry modules (E_z, D_z) and (E_a, D_a) (Stage 2). The Stage 2 symmetry modules are two residual MLP autoencoders that wrap the Stage 1 perceptual latent and the environment action in their own learnable spherical symmetry latent spaces. The observation symmetry encoder E_z maps the Stage 1 perceptual latent $z \in \mathbb{S}^{15}$ to an observation symmetry latent $u = E_z(z)$ with dimension $d_u = 64$, and the observation symmetry decoder D_z maps it back to $\mathbb{S}^{15}$. The action symmetry encoder E_a maps each normalized environment action $a \in \mathcal{A} \subseteq [-1, 1]^{d_a}$ to an action symmetry latent $v = E_a(a)$ with dimension $d_v = 16$, and the action symmetry decoder D_a maps it back to normalized action coordinates.

All four modules share a common MLP template. Each encoder consists of a linear projection $\mathrm{Linear}(d_{\mathrm{in}}, 128)$, a single width-128 residual block, and a final GELU–$\mathrm{Linear}(128, d_{\mathrm{lat}})$ projection. Each decoder is wider: a linear expansion $\mathrm{Linear}(d_{\mathrm{lat}}, 256)$, a single width-256 residual block, and a final GELU–$\mathrm{Linear}(256, d_{\mathrm{out}})$ projection. The observation branch uses $d_{\mathrm{in}} = d_{\mathrm{out}} = d_z$ and $d_{\mathrm{lat}} = d_u$; the action branch uses $d_{\mathrm{in}} = d_{\mathrm{out}} = d_a$ and $d_{\mathrm{lat}} = d_v$. Every residual block applies $\mathrm{LayerNorm}$–GELU–Linear–$\mathrm{LayerNorm}$–GELU–Linear with an additive skip connection, and we use neither spectral normalization nor orthogonal parametrization.

Normalization and initialization. All Stage 2 symmetry latents are projected onto the unit hypersphere, and the output of D_z is additionally projected back onto the Stage 1 perceptual latent space $\mathbb{S}^{15}$. Keeping transformed perceptual latents on the same sphere as the original latents facilitates adversarial tuple matching. Linear layers use Kaiming initialization tuned for GELU activations, except for the final encoder projection, which uses Xavier-normal initialization with gain 0.01 and zero bias. The decomposition of each symmetry latent into rotating and invariant coordinates is specified in Section C.2.

Discriminator f_ψ. The tuple discriminator f_ψ is a spectral-normalized MLP with two scoring heads: a transition head f_ψ^{mdp} that scores latent transition tuples jointly and an auxiliary marginal head f_ψ^{obs} that scores perceptual latents on their own. The transition head ingests a latent transition tuple (z, a, z', r), where $z, z' \in \mathbb{S}^{15}$ are Stage 1 perceptual latents, $a \in \mathbb{R}^{d_a}$ is the environment action, and r is the per-task vector-valued reward defined in Appendix B.3. Each of the four components is first embedded into $\mathbb{R}^{32}$ by a dedicated spectral-normalized linear subnet, and the four

embeddings are concatenated to form a 128-dimensional tuple feature. This feature is then mapped to a scalar logit via three spectral-normalized linear layers, interleaved with leaky ReLU activations. The marginal head reuses the same backbone but starts from a single $\mathbb{R}^{128}$ embedding of a perceptual latent; we apply it independently to z and z' and concatenate the two scalar scores to form its output.

C.2 Fixed Group Representations

We instantiate ρ_z and ρ_a as fixed block-diagonal representations of $g \in SO(2)$ in the learned observation and action symmetry latents. The angle $\theta_g \in [0, 2\pi)$ parameterizes g, and $R(\theta_g) \in \mathbb{R}^{2\times 2}$ is the standard 2D rotation matrix. For either input branch, indexed by $\bullet \in \{z, a\}$, the representation rotates $K_\bullet$ two-dimensional blocks and preserves $M_\bullet$ invariant coordinates:

$$\rho_\bullet(g) = \mathrm{diag}\big(\underbrace{R(\theta_g), \ldots, R(\theta_g)}_{K_\bullet \text{ copies}}, I_{M_\bullet}\big).$$

Thus, $d_u = 2K_z + M_z$ and $d_v = 2K_a + M_a$, with the dimensions listed in Table 4.

Table 4: Decomposition of the Stage 2 symmetry latents. Each rotating block has two coordinates.

Symmetry latent	Rotating blocks $K_\bullet$	Invariant coordinates $M_\bullet$	Total dimension
Observation u	16	32	$d_u = 64$
Action v	4	8	$d_v = 16$

Decoding the transformed symmetry latents gives the operations

$$\phi_z(g, z) = D_z\big(\rho_z(g)E_z(z)\big), \qquad \phi_a(g, a) = D_a\big(\rho_a(g)E_a(a)\big).$$

Here, ϕ_z acts on the Stage 1 perceptual latent, while ϕ_a returns actions in the normalized coordinates defined in Appendix B.2. The corresponding observation-space operation is $\phi_o(g, o) = D_o(\phi_z(g, E_o(o)))$. As in Section 4.1, the angle specifies a rotation in the learned symmetry latents; the induced nonlinear maps ϕ_o and ϕ_a need not be geometric rotations by the same angle.

C.3 Training Objectives and Procedure

Stage 1: Perceptual pretraining. We train (E_o, D_o) on observations from $\mathcal{D}$ with a normalized L_1 reconstruction loss after an initial L_2 warm-up. A latent cycle-consistency term encourages $E_o(D_o(z)) \approx z$. When enabled, a perceptual discriminator supplies adversarial and feature-matching losses to sharpen reconstruction texture; their weights are introduced with a linear warm-up. The warm-up durations and optimization settings are collected in Section C.4.

Transition to Stage 2. After pretraining, we freeze (E_o, D_o) and encode each transition tuple $\tau = (o, a, o', r)$ from the offline dataset as a latent transition tuple $t = (z, a, z', r)$ with $z = E_o(o)$ and $z' = E_o(o')$. Stage 2 trains the symmetry modules (E_z, D_z, E_a, D_a) and the tuple discriminator f_ψ. For a sampled g, the transformed tuple $\Phi_z(g, t)$ applies ϕ_z to both perceptual latents and ϕ_a to the action, leaving the reward unchanged.

Stage 2: Adversarial tuple matching and group-consistency regularization. We implement the adversarial tuple-matching objective of Eq. (1) as a multi-head hinge-GAN on raw logits, applied to each scoring head as an independent discriminator and summed in a multi-discriminator fashion. Concretely, the transition head contributes the hinge discriminator loss

$$\mathcal{L}^{\mathrm{mdp}}_{\mathrm{disc}} \;=\; \tfrac{1}{2}\,\mathbb{E}_t\Big[\max(0,\, 1 - f^{\mathrm{mdp}}_\psi(t))\Big] \;+\; \tfrac{1}{2}\,\mathbb{E}_{t,g}\Big[\max(0,\, 1 + f^{\mathrm{mdp}}_\psi(\Phi_z(g, t)))\Big],$$

and the marginal head contributes the same hinge form applied independently to each perceptual latent in the tuple, treating z and $\phi_z(g, z)$ as a real/fake pair and likewise for z' and $\phi_z(g, z')$. The discriminator minimizes the total discriminator loss $\mathcal{L}_{\mathrm{disc}} = \mathcal{L}^{\mathrm{mdp}}_{\mathrm{disc}} + \mathcal{L}^{\mathrm{obs}}_{\mathrm{disc}}$. The symmetry

modules act as the generator and minimize $\mathcal{L}_{\text{gen}} = -\mathbb{E}_{t,g}[f_\psi(\Phi_z(g,t))]$, summed across heads in the same multi-head fashion. This loss is the hinge realization of the $\mathcal{L}_{\text{adv}}$ term that enters the symmetry modules' total loss in Eq. (3). The symmetry modules additionally minimize the identity, compatibility, and invertibility regularizers of Eq. (2), and the nontriviality penalty $\mathcal{L}_{\text{nt}}$ between z and $\phi_z(g,z)$ and between z' and $\phi_z(g,z')$. Their weighted sum with the adversarial tuple-matching loss is specified in Eq. (3).

Input normalization and alternating updates. Perceptual latents and actions are passed to f_ψ without further normalization, since the latents already lie on $\mathbb{S}^{15}$ by construction and the environment actions are already normalized to $[-1,1]^{d_a}$; reward components are standardized to zero mean and unit variance across $\mathcal{D}$. Stabilization relies on the discriminator's spectral normalization together with gradient clipping; we use neither a gradient penalty nor an R_1 regularizer. The discriminator and the symmetry modules are updated one-to-one: one discriminator step follows every symmetry-module step.

Training-time group sampling. We draw rotation angles as $\theta_g \sim \mathcal{N}(0,\pi^2)$, interpreted modulo 2π. This sampler has support over the full circle and includes no reflections, consistent with $G = SO(2)$. Evaluation-time group sampling is specified separately in Appendix B.5.3.

C.4 Training Hyperparameters

Table 5 collects the core optimization settings for both stages. The Stage 1 autoencoder learning rate is held constant throughout training, and both the Stage 2 symmetry modules and tuple discriminator use Adam betas $(0.5, 0.999)$.

Table 5: Core symmetry-discovery optimization settings. Stage 2 settings apply to both the symmetry modules and tuple discriminator, except that the listed batch size is for symmetry-module updates.

Setting	Stage 1 Perceptual autoencoder	Stage 2 Symmetry discovery
Optimizer	Adam	Adam
Learning rate	2×10^{-4}	1×10^{-4}
Gradient updates	100,000	100,000
Batch size	32	4096
Weight decay	0	0
Gradient clipping (max. norm)	1.0	1.0

Stage 1 warm-up and auxiliary losses. The reconstruction loss uses L_2 for the first 2,000 iterations before switching to normalized L_1. When the perceptual discriminator and feature-matching loss are enabled, the discriminator uses Adam with betas $(0.5, 0.999)$, batch size 32, and gradient clipping at norm 1.0. Its learning rate is 2×10^{-4} for RGB inputs and 1×10^{-4} for heightmap and segmentation-mask inputs. The adversarial and feature-matching loss weights increase linearly from zero to 0.1 and 2.0, respectively, over the first 2,000 iterations.

Stage 2 loss weights. The Stage 2 objective $\mathcal{L}_{\text{total}}$ (Eq. 3) uses $\lambda_{\text{adv}} = \lambda_{\text{id}} = \lambda_{\text{comp}} = \lambda_{\text{inv}} = 1.0$, applied without warm-up, and $\lambda_{\text{nt}} = 0.1$ for the nontriviality penalty.

Seeds. All reported MIRROR results are averaged over three random seeds (seed $\in \{1,2,3\}$).

D Additional Symmetry-Discovery Results

We first report per-task errors on the symmetry-quality metrics and ablation results, then examine their dependence on the sampled group elements. We conclude with visualizations of the coupled observation–action transformations and alignment of transition-tuple embeddings during training. All results follow the metric definitions and evaluation protocol in Appendix B.5.

D.1 Quantitative Comparison across Tasks

Tables 6–9 compare all methods on the four Robosuite tasks. Symmetry invariance and invertibility errors are averaged over 256 uniformly sampled angles in $SO(2)$; compatibility error is averaged over all 256 ordered pairs of 16 equally spaced angles; identity error is evaluated at $g = e$. Lower is better for all four metrics.

D.1.1 Symmetry invariance error

MIRROR attains the lowest aggregate symmetry invariance error and the best per-task means on Cube Dropping and Hammer Lifting (Table 6). SymGAN and w/o $\mathcal{L}_{\text{inv}}$ obtain the lowest means on Drawer Opening and Drawer Closing, respectively. Removing $\mathcal{L}_{\text{adv}}$ yields the largest error on every task.

Table 6: **Average symmetry invariance error** $\overline{\mathcal{E}}_{\text{sym}}$. Each task entry reports mean $\pm$ standard deviation across three training seeds, after averaging over 256 evaluation angles within each seed. The All Tasks row reports the mean and standard deviation across all twelve task–seed averages. Lower is better.

Task	MIRROR	SymGAN	LaLiGAN	w/o $\mathcal{L}_{\text{adv}}$	w/o $\mathcal{L}_{\text{id}}$	w/o $\mathcal{L}_{\text{comp}}$	w/o $\mathcal{L}_{\text{inv}}$
Drawer Opening	0.198 ± 0.021	$\mathbf{0.180 \pm 0.035}$	0.276 ± 0.060	1.794 ± 0.114	0.208 ± 0.021	0.223 ± 0.029	0.225 ± 0.011
Drawer Closing	0.174 ± 0.005	0.233 ± 0.042	0.498 ± 0.087	1.942 ± 0.035	0.171 ± 0.026	0.179 ± 0.003	$\mathbf{0.145 \pm 0.015}$
Cube Dropping	$\mathbf{0.124 \pm 0.023}$	0.213 ± 0.012	0.208 ± 0.009	1.522 ± 0.115	0.131 ± 0.023	0.136 ± 0.015	0.129 ± 0.013
Hammer Lifting	$\mathbf{0.131 \pm 0.011}$	0.322 ± 0.025	0.336 ± 0.051	1.546 ± 0.035	0.137 ± 0.005	0.159 ± 0.002	0.158 ± 0.004
All Tasks	$\mathbf{0.157 \pm 0.035}$	0.237 ± 0.061	0.329 ± 0.122	1.701 ± 0.195	0.162 ± 0.037	0.174 ± 0.036	0.164 ± 0.038

D.1.2 Identity error

Removing $\mathcal{L}_{\text{id}}$ increases identity error on all four tasks (Table 7). Among MIRROR, SymGAN, and LaLiGAN, LaLiGAN has the lowest aggregate identity error. The w/o $\mathcal{L}_{\text{adv}}$ ablation obtains the lowest values overall, despite its large errors on the other three metrics.

Table 7: **Average identity error** $\mathcal{E}_{\text{id}}$. Mean $\pm$ standard deviation across tasks. Lower is better.

Task	MIRROR	SymGAN	LaLiGAN	w/o $\mathcal{L}_{\text{adv}}$	w/o $\mathcal{L}_{\text{id}}$	w/o $\mathcal{L}_{\text{comp}}$	w/o $\mathcal{L}_{\text{inv}}$
Drawer Opening	0.295 ± 0.010	1.790 ± 0.062	0.317 ± 0.003	$\mathbf{0.157 \pm 0.007}$	0.361 ± 0.024	0.308 ± 0.003	0.322 ± 0.012
Drawer Closing	0.324 ± 0.022	2.049 ± 0.005	0.351 ± 0.021	$\mathbf{0.162 \pm 0.003}$	0.372 ± 0.072	0.321 ± 0.011	0.321 ± 0.043
Cube Dropping	0.344 ± 0.050	1.769 ± 0.018	0.331 ± 0.013	$\mathbf{0.195 \pm 0.015}$	0.440 ± 0.020	0.347 ± 0.007	0.377 ± 0.026
Hammer Lifting	0.303 ± 0.013	2.208 ± 0.176	0.185 ± 0.008	$\mathbf{0.111 \pm 0.001}$	0.757 ± 0.018	0.322 ± 0.004	0.326 ± 0.013
All Tasks	0.317 ± 0.034	1.954 ± 0.206	0.296 ± 0.066	$\mathbf{0.156 \pm 0.031}$	0.483 ± 0.166	0.325 ± 0.016	0.337 ± 0.036

D.1.3 Compatibility error

MIRROR achieves the lowest aggregate compatibility error and the best per-task means on Drawer Opening, Cube Dropping, and Hammer Lifting (Table 8). The w/o $\mathcal{L}_{\text{inv}}$ ablation obtains the lowest mean on Drawer Closing. Removing $\mathcal{L}_{\text{comp}}$ increases compatibility error on every task.

Table 8: **Average compatibility error** $\overline{\mathcal{E}}_{\text{comp}}$. Mean $\pm$ standard deviation across tasks. Lower is better.

Task	MIRROR	SymGAN	LaLiGAN	w/o $\mathcal{L}_{\text{adv}}$	w/o $\mathcal{L}_{\text{id}}$	w/o $\mathcal{L}_{\text{comp}}$	w/o $\mathcal{L}_{\text{inv}}$
Drawer Opening	$\mathbf{0.856 \pm 0.283}$	1.824 ± 0.136	1.853 ± 0.155	3.531 ± 0.192	1.009 ± 0.221	1.308 ± 0.276	1.090 ± 0.094
Drawer Closing	0.668 ± 0.059	2.121 ± 0.109	1.959 ± 0.149	3.314 ± 0.120	0.651 ± 0.292	0.871 ± 0.030	$\mathbf{0.445 \pm 0.155}$
Cube Dropping	$\mathbf{0.521 \pm 0.160}$	1.888 ± 0.052	1.679 ± 0.093	3.718 ± 0.364	0.662 ± 0.074	0.913 ± 0.204	0.614 ± 0.272
Hammer Lifting	$\mathbf{0.821 \pm 0.123}$	2.144 ± 0.016	1.861 ± 0.007	3.012 ± 0.118	0.951 ± 0.078	1.443 ± 0.025	1.101 ± 0.038
All Tasks	$\mathbf{0.717 \pm 0.221}$	1.994 ± 0.167	1.838 ± 0.155	3.394 ± 0.344	0.818 ± 0.251	1.134 ± 0.301	0.813 ± 0.333

D.1.4 Invertibility error

MIRROR attains the lowest aggregate invertibility error and the best mean on Cube Dropping (Table 9). SymGAN, w/o $\mathcal{L}_{\text{inv}}$, and w/o $\mathcal{L}_{\text{comp}}$ obtain the lowest means on Drawer Opening, Drawer Closing, and Hammer Lifting, respectively. Removing $\mathcal{L}_{\text{inv}}$ increases invertibility error on three tasks but reduces it on Drawer Closing.

Table 9: **Average invertibility error** $\overline{\mathcal{E}}_{\text{inv}}$. Mean $\pm$ standard deviation across tasks. Lower is better.

Task	MIRROR	SymGAN	LaLiGAN	w/o $\mathcal{L}_{\text{adv}}$	w/o $\mathcal{L}_{\text{id}}$	w/o $\mathcal{L}_{\text{comp}}$	w/o $\mathcal{L}_{\text{inv}}$
Drawer Opening	0.911 ± 0.242	$\mathbf{0.649 \pm 0.095}$	1.138 ± 0.063	5.947 ± 0.369	1.051 ± 0.200	1.083 ± 0.347	1.115 ± 0.105
Drawer Closing	0.712 ± 0.057	1.132 ± 0.281	1.927 ± 0.208	6.142 ± 0.211	0.689 ± 0.229	0.652 ± 0.006	$\mathbf{0.537 \pm 0.154}$
Cube Dropping	$\mathbf{0.612 \pm 0.132}$	1.498 ± 0.152	1.360 ± 0.114	4.594 ± 0.324	0.771 ± 0.065	0.735 ± 0.099	0.747 ± 0.205
Hammer Lifting	0.685 ± 0.081	1.514 ± 0.004	1.480 ± 0.056	3.842 ± 0.185	0.744 ± 0.053	$\mathbf{0.683 \pm 0.016}$	0.903 ± 0.018
All Tasks	$\mathbf{0.730 \pm 0.183}$	1.198 ± 0.389	1.476 ± 0.314	5.131 ± 0.995	0.814 ± 0.211	0.788 ± 0.250	0.826 ± 0.253

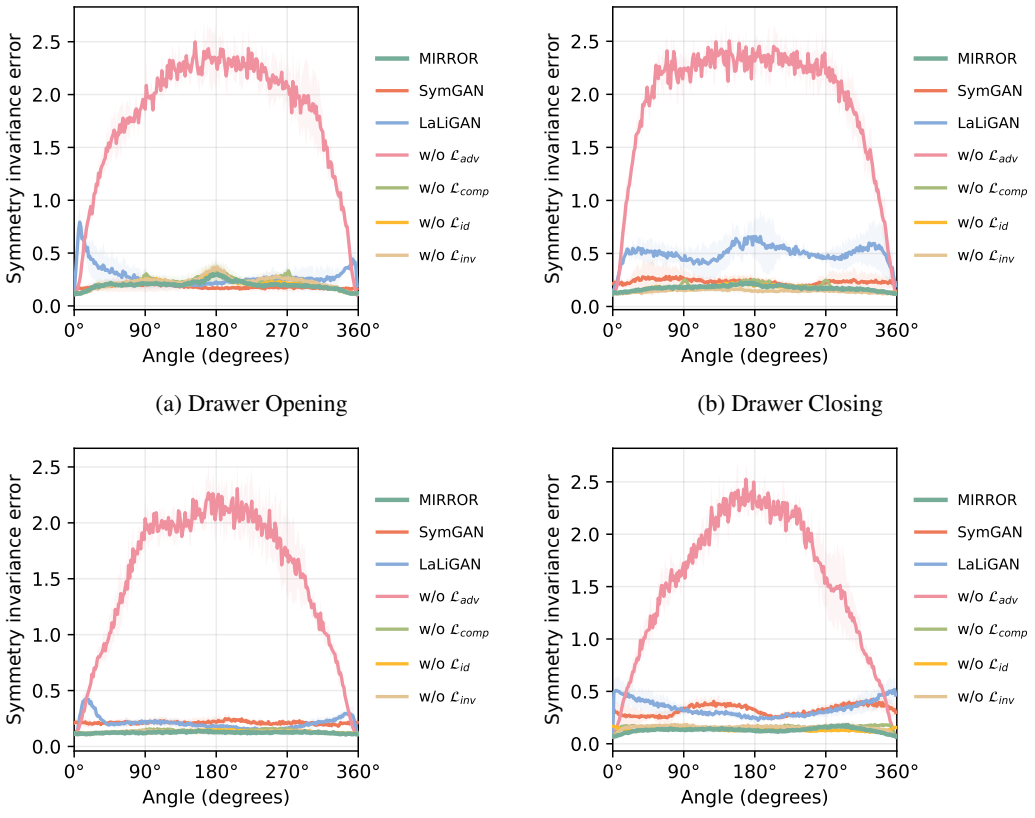
Joint interpretation. MIRROR has the lowest aggregate symmetry invariance, compatibility, and invertibility errors. Removing the adversarial tuple-matching term improves identity but substantially worsens the other three metrics. Removing compatibility increases aggregate compatibility error more than removing identity or invertibility, consistent with Appendix A.2. The task-level exceptions show that the benefits of individual regularizers depend on the task.

D.2 Symmetry Quality across Group Elements

The curves and heatmaps use the same evaluation angles and angle pairs as Section D.1, revealing whether errors concentrate near particular transformations.

D.2.1 Symmetry invariance across angles

Figure 9 shows low, nearly flat symmetry invariance error for MIRROR across the sampled angles on all four tasks. Removing $\mathcal{L}_{\text{adv}}$ produces broad plateaus or peaks near $180°$, revealing distribution mismatch across a wide range of transformations.



(a) Drawer Opening

(b) Drawer Closing

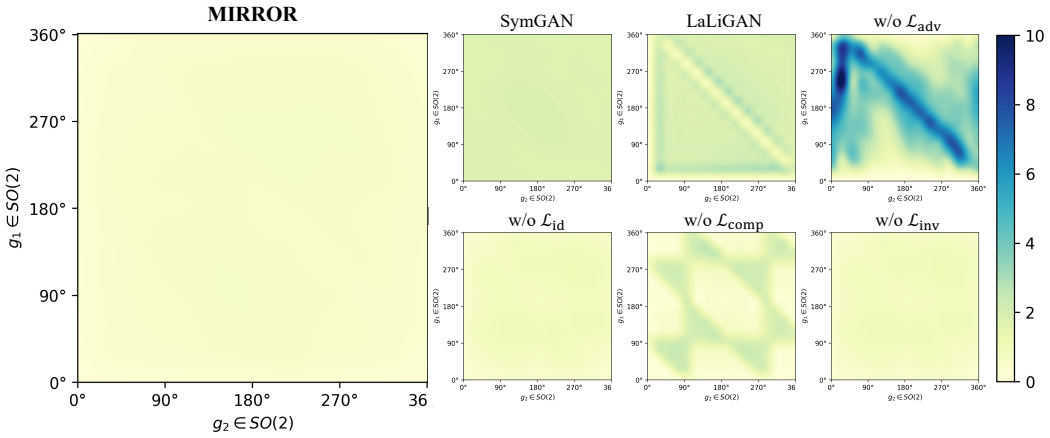
(c) Cube Dropping

(d) Hammer Lifting

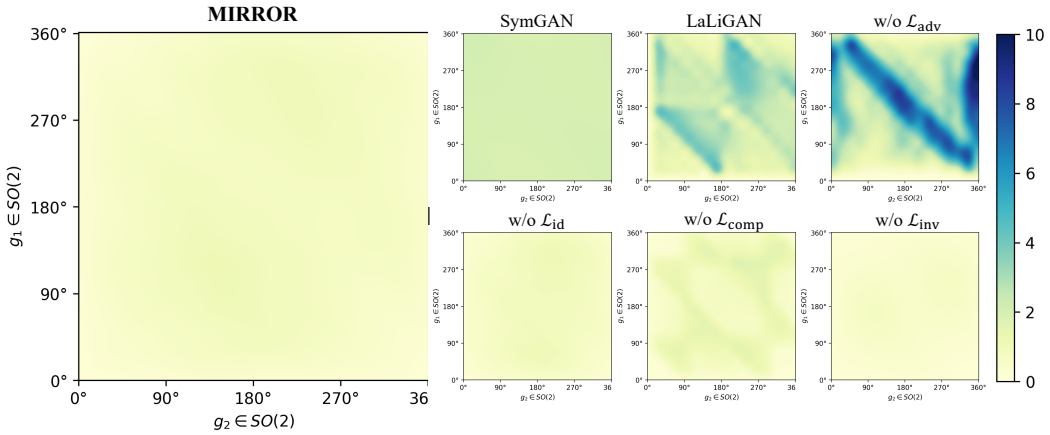
Figure 9: **Symmetry invariance error across latent rotation angles.** Lines show the mean feature-space Wasserstein-2 distance between original and transformed transition-tuple distributions at 256 uniformly sampled $SO(2)$ angles. Shaded bands indicate ± 1 standard deviation across three seeds; lower is better. MIRROR maintains low error across all four tasks.

D.2.2 Compatibility across Angle Pairs

Figure 10 shows low compatibility error for MIRROR across the evaluated angle pairs. Removing $\mathcal{L}_{\text{adv}}$ produces bands near inverse-angle pairs $\theta_{g_2} \approx -\theta_{g_1} \pmod{2\pi}$ or high-error regions near $\theta_{g_1} = 180°$. LaLiGAN shows narrower ridges or intersecting streaks, while SymGAN has more uniformly elevated error.

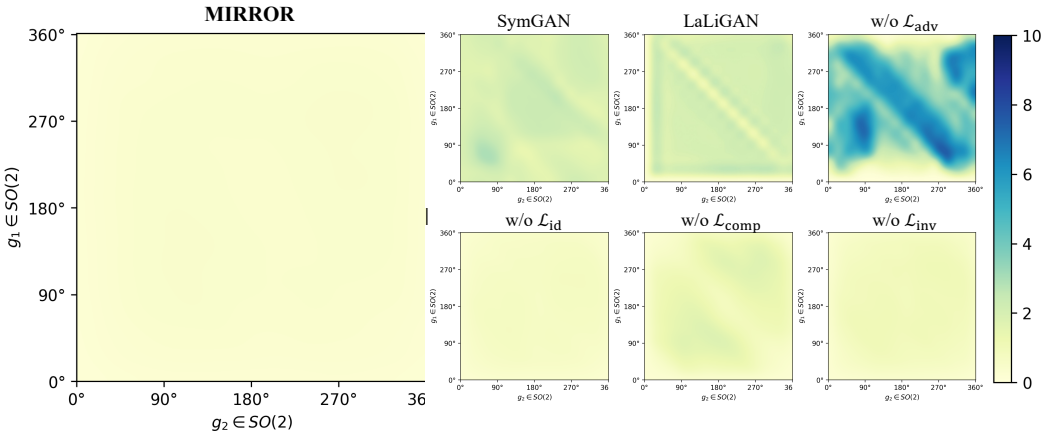


(a) Drawer Opening

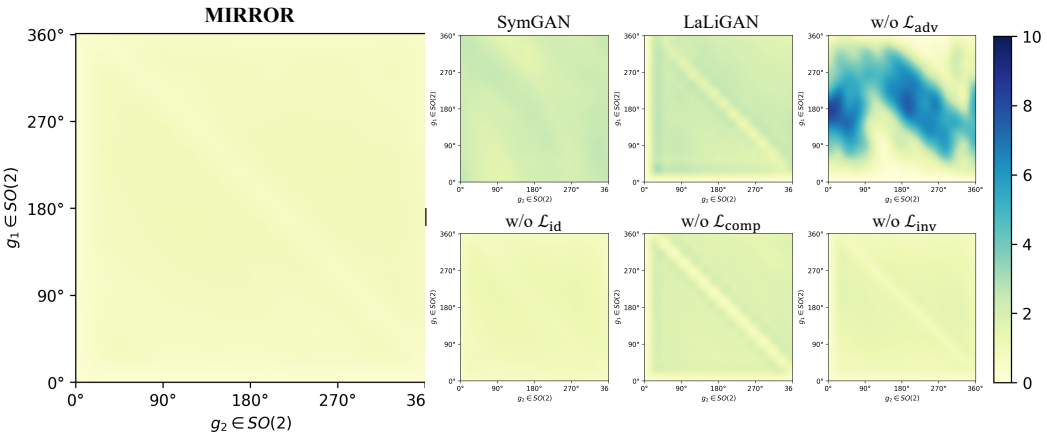


(b) Drawer Closing

Figure 10: **Compatibility error heatmaps across four tasks.** Compatibility error compares a transition tuple transformed successively by g_1 and g_2 with the same tuple transformed once by their composition $g_2 \circ g_1$. Errors are evaluated at all 256 ordered pairs of 16 equally spaced rotation angles; the heatmaps are smoothed for visualization. Within each task panel, the large left subplot shows MIRROR and the six smaller subplots show baselines and single-loss ablations on a shared color scale (lower is better). MIRROR maintains low error across the evaluated angle pairs. Panels (c)–(d) continue on the next page.



(c) Cube Dropping

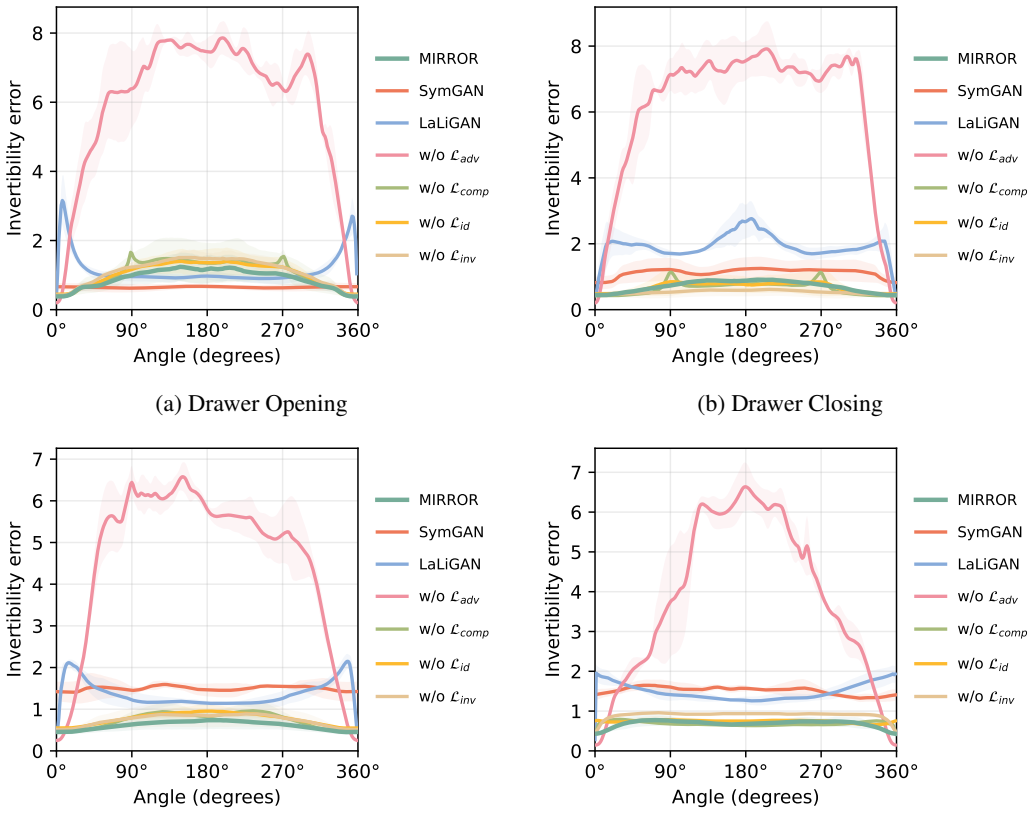


(d) Hammer Lifting

Figure 10: **Compatibility error heatmaps across four tasks (continued).** Panels (c)–(d) show Cube Dropping and Hammer Lifting. The smoothed heatmaps compare successive transformations by g_1 and g_2 with their composition $g_2 \circ g_1$, evaluated at all 256 ordered pairs of 16 equally spaced rotation angles. Within each task panel, MIRROR, baselines, and single-loss ablations share a color scale. Lower is better.

D.2.3 Invertibility across Angles

Figure 11 shows that MIRROR maintains relatively low invertibility error across the sampled angles on all four tasks. Removing $\mathcal{L}_{\text{adv}}$ produces broad plateaus or peaks near $180°$, indicating that invertibility also degrades across a wide range of transformations when adversarial tuple matching is removed. The remaining methods exhibit task-dependent differences, as summarized in Table 9.



(a) Drawer Opening

(b) Drawer Closing

(c) Cube Dropping

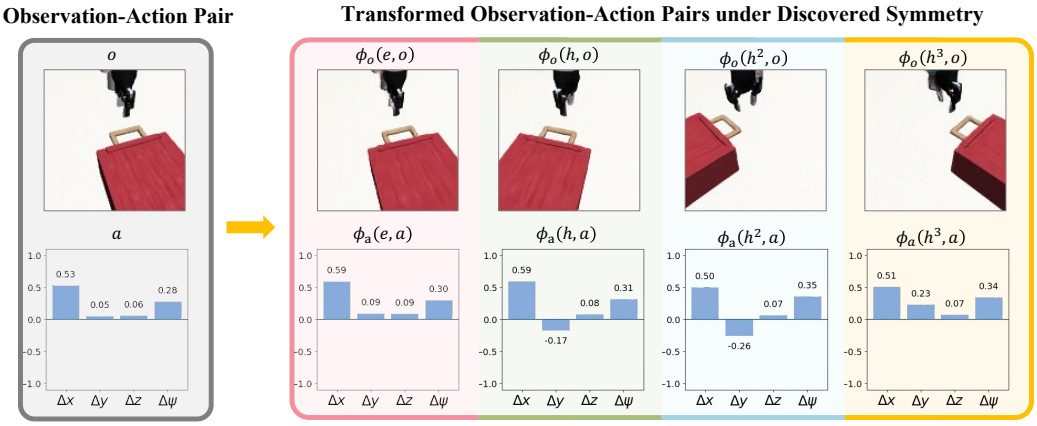
(d) Hammer Lifting

Figure 11: **Invertibility error across latent rotation angles.** For each task, $\mathcal{E}_{\text{inv}}(g)$ measures how closely applying g followed by g^{-1} recovers the original transition tuple. Lines show mean errors at 256 angles uniformly sampled from $SO(2)$, and shaded bands indicate ± 1 standard deviation across three training seeds. Lower is better. MIRROR maintains relatively low recovery error across the sampled angles on all four tasks.

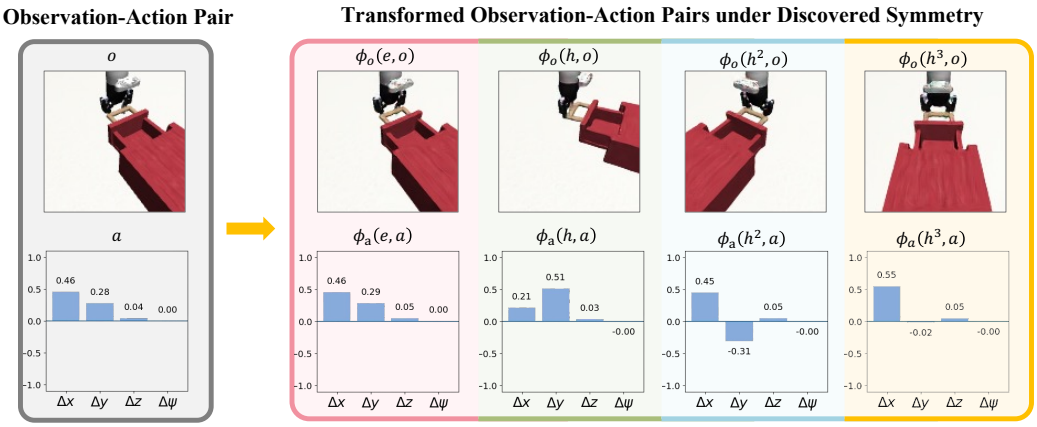
D.3 Qualitative Visualization of Discovered Symmetries

D.3.1 Learned Observation–Action Transformations

Figure 12 applies each $g \in C_4$ to the same observation–action pair (o, a) in each environment. Across all four tasks, $\phi_o(g, o)$ changes the scene configuration while $\phi_a(g, a)$ adjusts the normalized planar commands $(\Delta x, \Delta y)$ jointly with it. The remaining action components (Δz and the task-specific $\Delta\psi$ and/or Δw) remain approximately unchanged. The angles specify rotations in the learned symmetry latents; their geometric effect after decoding is determined by the learned maps.

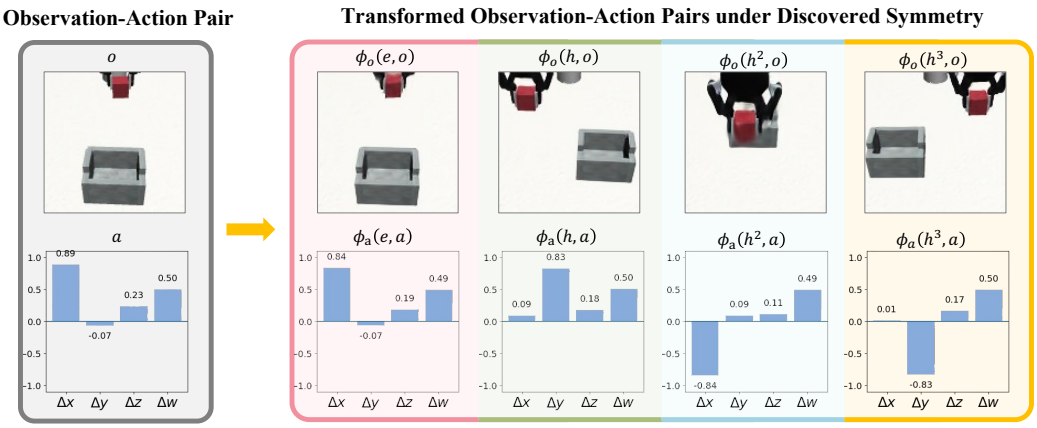


(a) Drawer Opening

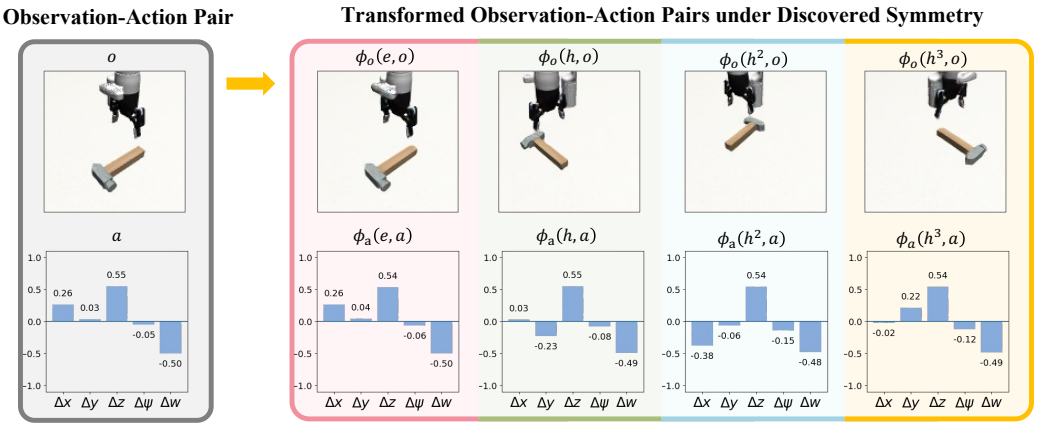


(b) Drawer Closing

Figure 12: **Learned RGB observation–action transformations.** Each task panel shows a fixed original pair (o, a) on the left and four transformed pairs $(\phi_o(g, o), \phi_a(g, a))$ on the right, for $g \in C_4 = \{e, h, h^2, h^3\}$ with latent angles $0°, 90°, 180°, 270°$, respectively. Action bars show normalized command components on a $[-1, 1]$ scale. The scene and planar commands $(\Delta x, \Delta y)$ change jointly; the decoded transformations need not be geometric rotations by the latent angles. Panels (c)–(d) continue on the next page.



(c) Cube Dropping

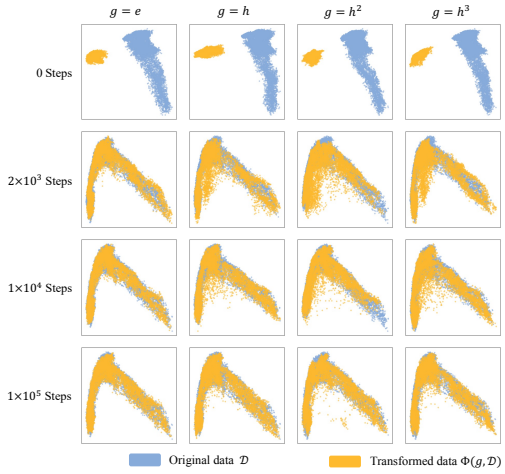


(d) Hammer Lifting

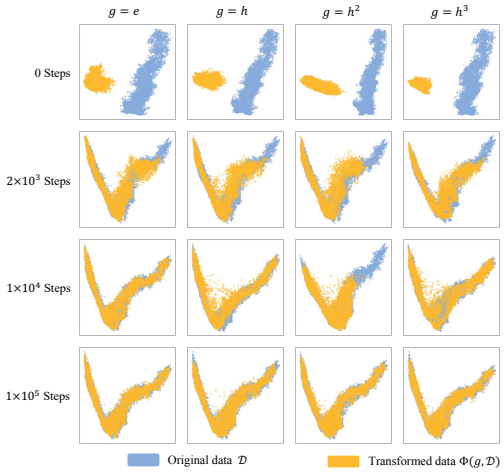
Figure 12: **Learned RGB observation–action transformations (continued).** Panels (c)–(d) show Cube Dropping and Hammer Lifting, with the original pair on the left and four transformed pairs on the right. Columns e, h, h^2, h^3 correspond to latent angles $0°, 90°, 180°, 270°$, respectively; decoded transformations need not be geometric rotations by these angles. Action bars show normalized command components on a $[-1, 1]$ scale, with Δw denoting gripper opening.

D.3.2 Alignment of Transition-Tuple Embeddings during Training

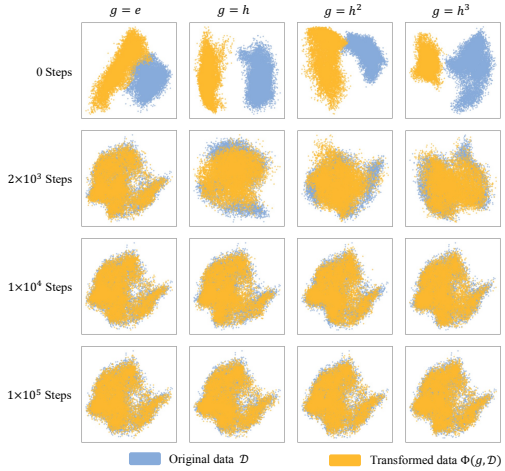
Figure 13 projects transition-tuple embeddings of $\mathcal{D}$ and $\Phi(g, \mathcal{D})$ to two dimensions with PCA, separately for each $g \in C_4$ and at four training checkpoints. The projected clouds are initially separated for nonidentity elements and increasingly overlap during training. By 1×10^5 steps, they overlap substantially across the four tasks. This visualization supports the quantitative distribution-matching results in Table 6; overlap in a two-dimensional projection alone does not establish equality of the full transition-tuple distributions.



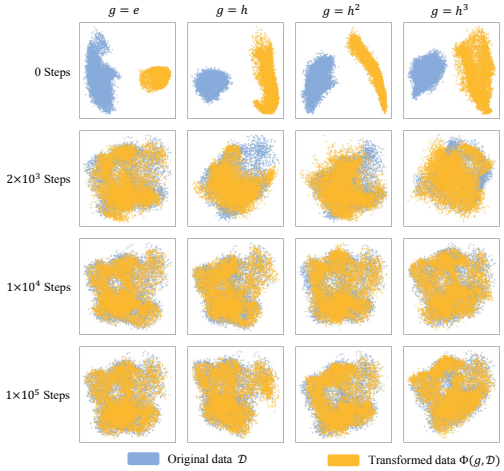
(a) Drawer Opening



(b) Drawer Closing



(c) Cube Dropping



(d) Hammer Lifting

Figure 13: **Alignment of transition-tuple embeddings during training.** Two-dimensional PCA projections show embeddings of the original dataset $\mathcal{D}$ (blue) and transformed dataset $\Phi(g, \mathcal{D})$ (orange) across four tasks. Columns correspond to $g \in C_4 = \{e, h, h^2, h^3\}$, where h denotes a 90° latent rotation; rows show 0, 2×10^3, 1×10^4, and 1×10^5 training steps. Increasing overlap qualitatively supports distribution matching.

E Frame-Averaged Behavior Cloning

This appendix describes EBC-MIRROR, the frame-averaged BC policy built on the discovered transformations (ϕ_o, ϕ_a). We present the policy construction and its equivariance conditions (Section E.1), architecture and training objective (Section E.2), baselines and evaluation protocol (Section E.3), and subgroup-size results (Section E.4). The RGB results support Section 5.3; the sim-to-real experiments are collected in Appendix F.

E.1 Frame-Averaged Policy and Equivariance Conditions

Let H be a finite subgroup of G and let f_θ be a base policy network that maps observations to actions. Following the frame-averaging construction [23], we use the discovered transformations ϕ_o and ϕ_a to define

$$\pi_\theta(o) \;:=\; \frac{1}{|H|}\sum_{g\in H} \phi_a\big(g^{-1},\, f_\theta(\phi_o(g,o))\big)\,. \tag{20}$$

Intuitively, we (i) transform the input observation by g, (ii) predict an action in that transformed frame via f_θ, (iii) map the predicted action back with $\phi_a(g^{-1},\cdot)$, and finally (iv) average over $g \in H$.

Our experiments use $G = SO(2)$ and average over a finite cyclic subgroup $H = C_n \subset SO(2)$. The equivariance result below concerns this subgroup H.

Conditions for exact equivariance. Assume that ϕ_o and ϕ_a satisfy the group-operation axioms exactly on the relevant supports, and that all transformations and averages below are well-defined. We additionally require the action transformation to commute with arithmetic averaging: for every $h \in H$ and admissible collection of actions $\{a_k\}_{k\in H}$,

$$\phi_a\left(h, \frac{1}{|H|}\sum_{k\in H} a_k\right) = \frac{1}{|H|}\sum_{k\in H} \phi_a(h, a_k). \tag{21}$$

This condition holds, for example, when each $\phi_a(h,\cdot)$ is affine and the averages remain in the action domain. Under these assumptions, reindexing the sum by $k = g \circ h$ gives

$$\begin{aligned}
\pi_\theta(\phi_o(h,o)) &= \frac{1}{|H|}\sum_{g\in H} \phi_a\big(g^{-1}, f_\theta(\phi_o(g\circ h, o))\big) \\
&= \frac{1}{|H|}\sum_{k\in H} \phi_a\big(h, \phi_a\big(k^{-1}, f_\theta(\phi_o(k,o))\big)\big) \\
&= \phi_a(h, \pi_\theta(o)).
\end{aligned}$$

The first two equalities use the exact group-operation axioms, and the last uses Eq. (21). Thus, Eq. (20) defines an H-equivariant policy under the stated conditions.

Scope for learned transformations. In MIRROR, $\phi_a(g,\cdot) = D_a(\rho_a(g)E_a(\cdot))$ is nonlinear, and the learned coupled observation–action transformations are regularized to satisfy the group-operation axioms. Neither exact group consistency nor Eq. (21) is guaranteed by this parameterization. We assess the resulting frame-averaged BC policy, EBC-MIRROR, through downstream policy success rates; the exact-equivariance guarantee above applies under its stated assumptions.

E.2 Policy Architecture and Training Objective

We instantiate f_θ with a pretrained ResNet-18 backbone followed by a two-layer MLP with 64 hidden units per layer and ReLU activations. The same base policy is applied to every transformed observation in Eq. (20).

During BC training, we hold the perceptual autoencoder and the observation and action symmetry modules fixed and optimize the base-policy parameters θ. The mean-squared BC loss is applied to the final averaged action:

$$\min_\theta\; \mathcal{L}_{\mathrm{EBC}}(\theta) \;:=\; \mathbb{E}_{(o,a)\sim\mathcal{D}}\Big[\, \|\pi_\theta(o) - a\|_2^2 \,\Big]. \tag{22}$$

Gradients propagate through the fixed action transformations to train f_θ within the frame-averaged policy.

E.3 Baselines and Evaluation Protocol

We evaluate policies by rolling them out on the four RGB Robosuite tasks described in Appendix B and report per-task episodic success rates and their average. We compare vanilla **BC**, which omits frame averaging, with four frame-averaged BC variants that use different coupled observation–action transformations. **EBC-MIRROR** uses our discovered transformations; **EBC-SymGAN** and **EBC-LaLiGAN** reuse the GAN-discovered transformations from Section 5; and **EBC-ExSym** uses the hand-designed *Extrapolated Symmetry* [24]. ExSym rotates images in pixel space and applies the following transformation directly to normalized action coordinates:

$$(\Delta x, \Delta y, \Delta z, \Delta\psi, \Delta w) \;\mapsto\; \big(R(\theta_g)(\Delta x, \Delta y)^\top,\, \Delta z,\, \Delta\psi,\, \Delta w\big),$$

where $R(\theta_g)$ is the 2D rotation matrix. This rotates only the planar translation components; $\Delta\psi$ or Δw is omitted when the task has no yaw or gripper control. For all frame-averaged BC variants we sweep the subgroup size $n = |H|$ with $H = C_n$ and $n \in \{2, 4, 8, 16\}$.

E.4 Effect of Subgroup Size

Table 10 reports the complete RGB subgroup sweep for $H \in \{C_2, C_4, C_8, C_{16}\}$, including the C_{16} results from main paper Table 2. The average success rate of EBC-MIRROR increases from 80.19% at C_2 to 81.91% at C_4, 83.57% at C_8, and 85.12% at C_{16}, exceeding vanilla BC (80.48%) for $|H| \geq 4$. Thus, C_{16} achieves the highest average success rate among the tested subgroup sizes, although the best subgroup varies by task.

Averaging over more group elements requires more transformed policy evaluations per action. Its performance benefit also depends on the transformations: EBC-ExSym, EBC-SymGAN, and EBC-LaLiGAN remain below vanilla BC on average at every tested subgroup size. These comparisons support the utility of MIRROR's learned transformations for frame averaging.

Table 10: Episodic success rates in percent (mean ± std) for $H \in \{C_2, C_4, C_8, C_{16}\}$, with vanilla BC as reference. The C_{16} rows reproduce main paper Table 2. **Bold** indicates the highest mean in each column across all listed methods and subgroup sizes, including BC.

Method	H	**Drawer Opening**	**Drawer Closing**	**Cube Dropping**	**Hammer Lifting**	**Average**
EBC-ExSym	C_2	62.5 ± 2.1	85.2 ± 1.4	79.5 ± 4.5	84.8 ± 4.3	78.0 ± 9.8
	C_4	61.5 ± 2.6	86.1 ± 3.5	83.5 ± 4.7	87.4 ± 2.2	79.6 ± 11.1
	C_8	62.1 ± 0.8	84.5 ± 5.0	79.5 ± 4.6	85.3 ± 5.6	77.9 ± 10.4
	C_{16}	64.7 ± 2.2	86.4 ± 4.7	80.9 ± 5.9	85.1 ± 5.7	79.3 ± 9.9
EBC-SymGAN	C_2	69.7 ± 7.5	87.0 ± 2.2	79.6 ± 7.9	81.7 ± 2.7	79.5 ± 8.5
	C_4	67.5 ± 8.0	86.6 ± 2.6	83.5 ± 4.0	68.1 ± 4.9	76.4 ± 10.2
	C_8	68.7 ± 8.7	83.7 ± 5.0	84.1 ± 5.1	57.5 ± 6.7	73.5 ± 12.9
	C_{16}	66.3 ± 11.3	85.8 ± 3.6	81.9 ± 9.4	49.8 ± 16.3	71.0 ± 18.1
EBC-LaLiGAN	C_2	69.0 ± 1.0	80.7 ± 6.6	85.1 ± 5.5	82.5 ± 0.8	79.3 ± 7.5
	C_4	69.0 ± 3.1	80.2 ± 8.9	79.5 ± 8.3	69.9 ± 10.1	74.7 ± 9.6
	C_8	67.9 ± 1.5	73.3 ± 3.0	83.1 ± 9.7	56.6 ± 2.4	70.2 ± 10.9
	C_{16}	65.3 ± 2.0	73.6 ± 6.2	84.5 ± 6.2	61.3 ± 5.6	71.2 ± 10.3
EBC-MIRROR	C_2	66.3 ± 1.6	$\mathbf{90.1 \pm 3.6}$	77.8 ± 4.2	86.5 ± 2.6	80.2 ± 9.7
	C_4	65.2 ± 2.5	86.9 ± 1.3	86.0 ± 3.5	$\mathbf{89.6 \pm 2.0}$	81.9 ± 10.1
	C_8	69.3 ± 3.1	90.0 ± 0.6	87.8 ± 3.5	87.2 ± 3.4	83.6 ± 8.8
	C_{16}	$\mathbf{71.7 \pm 1.4}$	89.8 ± 2.0	$\mathbf{90.3 \pm 2.9}$	88.7 ± 2.5	$\mathbf{85.1 \pm 8.1}$
BC	-	71.4 ± 0.8	88.1 ± 5.9	80.6 ± 2.5	81.8 ± 3.0	80.5 ± 7.0

F Sim-to-Real Experiments

This appendix details the sim-to-real experiments in Section 6. We describe the hardware and observation preprocessing, specify the training and deployment protocol, and then report simulation and real-world results. Qualitative visualizations of the learned observation–action transformations follow the quantitative evaluation.

F.1 Hardware and Observation Preprocessing

F.1.1 Hardware Setup

We deploy the learned policy on a 7-DoF Kinova GEN3 manipulator equipped with a Robotiq 2F85 parallel-jaw gripper. Visual observations are provided by an Intel RealSense D435 RGB-D camera mounted on a fixed stand in front of the workspace. The camera-to-robot extrinsics are obtained once via standard eye-to-hand calibration and reused across all tasks.

F.1.2 Observation Preprocessing

The policy consumes a heightmap and a target-object segmentation mask rather than raw RGB. Figure 14 illustrates the preprocessing pipeline, and main-paper Figure 8(e,f) compares the resulting real-world observation against its simulated counterpart.

Heightmap. We project the depth image into a top-down heightmap of the workspace and clip the height values to the range $[0,\ 0.3]$ m above the table surface. The clipped heights are then linearly rescaled to $[0,\ 255]$ and stored as an 8-bit single-channel image, matching the dynamic range used in simulation.

Segmentation mask. The target-object segmentation mask is obtained with SAM 2 [26], prompted using a height-derived foreground prior. The resulting binary segmentation mask is stacked with the heightmap to form the final policy input.

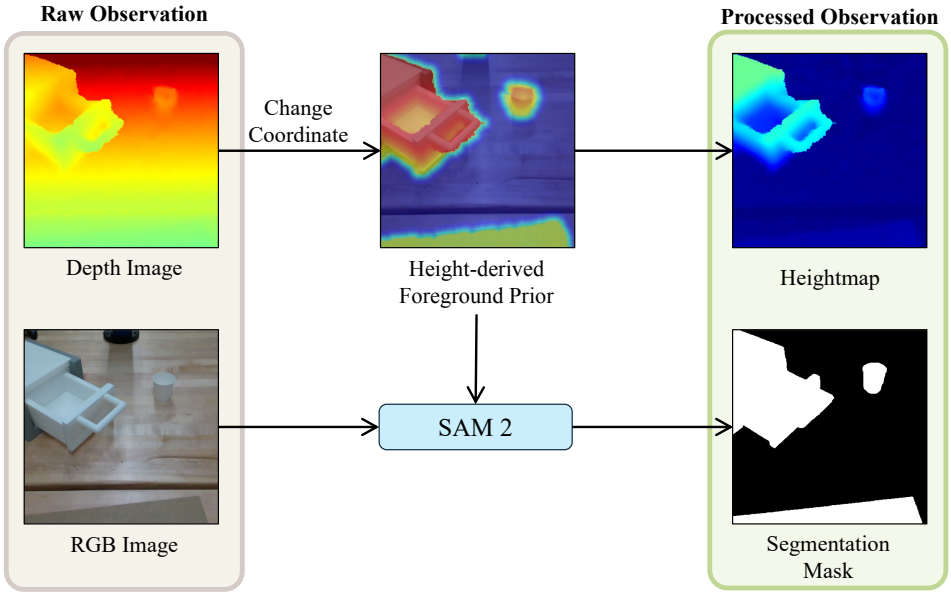


Figure 14: **Real-world observation preprocessing.** Depth is projected into a top-down heightmap, and a height-derived foreground prior prompts SAM 2 to segment the RGB image. The heightmap is clipped to $[0, 0.3]$ m above the table and rescaled to an 8-bit image, then stacked with the binary segmentation mask to form the two-channel policy input.

F.2 Training and Deployment Protocol

For real-world deployment, we use heightmap and segmentation-mask observations (Appendix F.1.2), similar to Lin et al. [27]. We run Stage 1 and Stage 2 symmetry discovery in simulation on this modality using the procedure in Appendix C, then freeze the resulting perceptual autoencoder and the observation and action symmetry modules.

We train a separate EBC-MIRROR policy entirely in simulation using these frozen transformations for frame averaging. The base policy architecture follows Appendix E.2, with its input encoder

adapted to the two-channel input, and the BC objective remains Eq. (22). Relative to the RGB experiments, the changes are the observation modality, the corresponding input adaptation, and the observation–action transformations learned for this modality.

The resulting policy is deployed *zero-shot* on the real robot using the hardware and preprocessing in Appendix F.1. We report simulation performance in Appendix F.3 and real-world outcomes in Appendix F.4.

F.3 Simulation Results with Heightmap and Segmentation-Mask Observations

We evaluate behavior-cloning policies in simulation using the same heightmap and segmentation-mask observation modality used at real-world deployment (Appendix F.1.2). This isolates the contribution of the discovered symmetry from the observation-modality change: every policy in Table 11 consumes the same heightmap and segmentation-mask input, and the only difference between rows is whether the discovered observation–action transformations (ϕ_o, ϕ_a) are used for frame-averaged BC (EBC-MIRROR, Appendix E.1) and, if so, with which finite cyclic subgroup $H = C_n$.

Each cell reports mean and standard deviation of episodic success rate over three seeds and the same evaluation protocol used in Appendix E.3.

Results. Table 11 shows that the trends observed under RGB observations carry over to the heightmap and segmentation-mask modality: increasing $|H|$ from C_2 to C_{16} monotonically raises the average success rate from 75.31% to 81.97%, and EBC-MIRROR with C_{16} attains the best average and the best per-task numbers on Drawer Opening and Cube Dropping. The vanilla BC baseline retains the lead on Drawer Closing, where the task is already largely saturated ($\approx 91\%$) and frame averaging offers little remaining headroom; EBC-MIRROR with $H = C_8$ marginally edges out the other rows on Hammer Lifting. Overall, all EBC-MIRROR configurations with $H \in \{C_4, C_8, C_{16}\}$ outperform the BC baseline on average, supporting the claim that MIRROR also discovers a usable $SO(2)$ symmetry under the deployment-time heightmap and segmentation-mask modality.

Table 11: **Simulated success rate with heightmap and segmentation-mask observations.** Episodic success rate (mean $\pm$ std over three seeds) on the heightmap and segmentation-mask input used at real-world deployment. We compare vanilla BC against EBC-MIRROR with $H = C_n$ for $n \in \{2, 4, 8, 16\}$. **Bold** indicates the best entry per column.

Method	H	**Drawer Opening**	**Drawer Closing**	**Cube Dropping**	**Hammer Lifting**	**Average**
EBC-MIRROR	C_2	0.5713 ± 0.0389	0.8843 ± 0.0416	0.7453 ± 0.1427	0.8113 ± 0.0413	0.7531 ± 0.1388
	C_4	0.5927 ± 0.0208	0.8971 ± 0.0539	0.8046 ± 0.0351	0.8331 ± 0.0804	0.7819 ± 0.1275
	C_8	0.6118 ± 0.0203	0.9070 ± 0.0359	0.8271 ± 0.0216	$\mathbf{0.8603 \pm 0.0631}$	0.8016 ± 0.1229
	C_{16}	$\mathbf{0.6694 \pm 0.0794}$	0.9050 ± 0.0380	$\mathbf{0.8536 \pm 0.0423}$	0.8507 ± 0.0253	$\mathbf{0.8197 \pm 0.1028}$
BC	-	0.6100 ± 0.0361	$\mathbf{0.9159 \pm 0.0075}$	0.7266 ± 0.0825	0.8566 ± 0.0903	0.7772 ± 0.1351

F.4 Real-World Validation

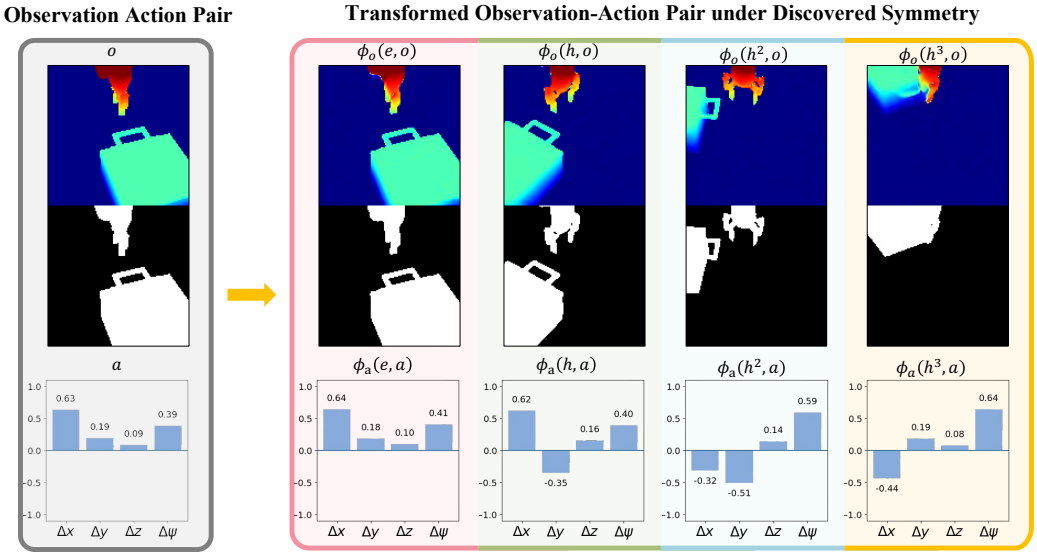
We evaluate the policy on the real robot using the hardware and preprocessing in Appendix F.1 and the training and deployment protocol in Appendix F.2. The four task setups are shown in Figure 8(a–d) of the main paper. Table 12 reports the success counts over 20 trials per task.

Table 12: Real-world task success counts over 20 trials per task.

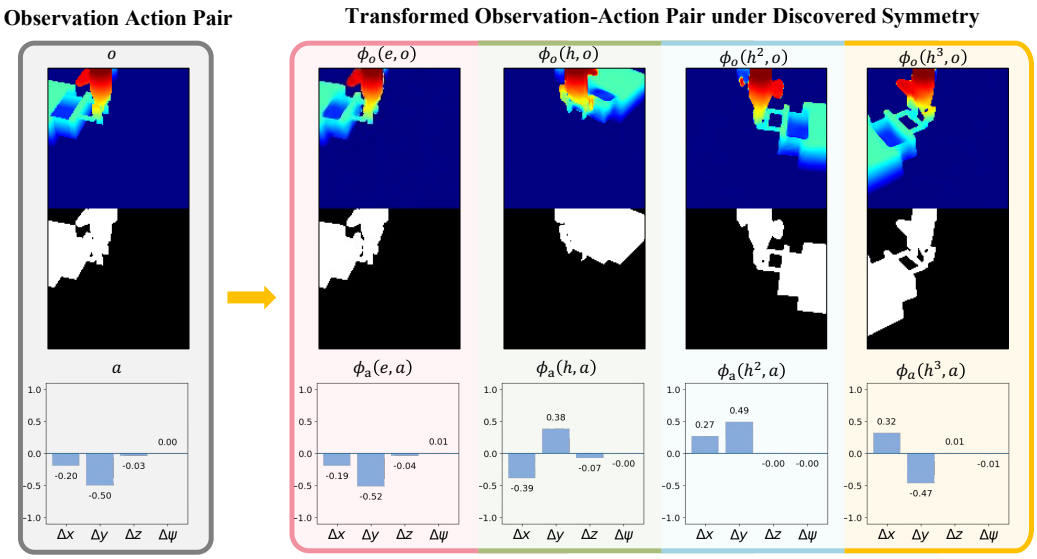
Task	Successful trials
Drawer Opening	13/20
Drawer Closing	19/20
Cube Dropping	9/20
Hammer Lifting	16/20

F.5 Qualitative Visualization of Learned Observation–Action Transformations

Figure 15 complements the policy results with visualizations of the coupled observation–action transformations for simulated heightmap and segmentation-mask observations. We show each $g \in C_4$ for all four manipulation tasks. In every environment, $\phi_o(g, o)$ transforms the top-down workspace and its accompanying segmentation mask jointly so that the target object remains correctly localized in the transformed observation. On the action side, $\phi_a(g, a)$ adjusts the normalized planar commands $(\Delta x, \Delta y)$ jointly with the observation transformation. As in the RGB experiments, g specifies a latent rotation whose effect in observation and action coordinates is learned.

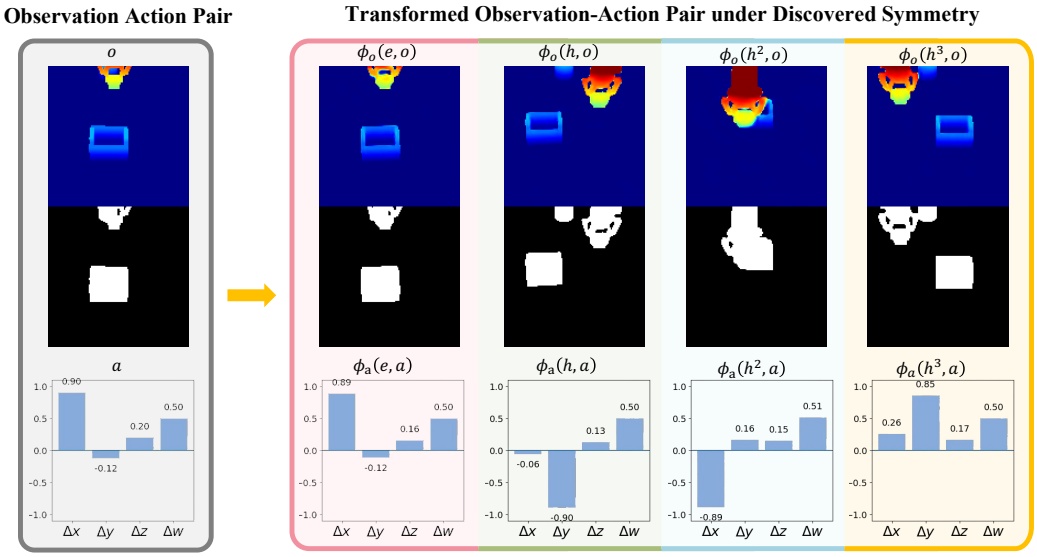


(a) Drawer Opening

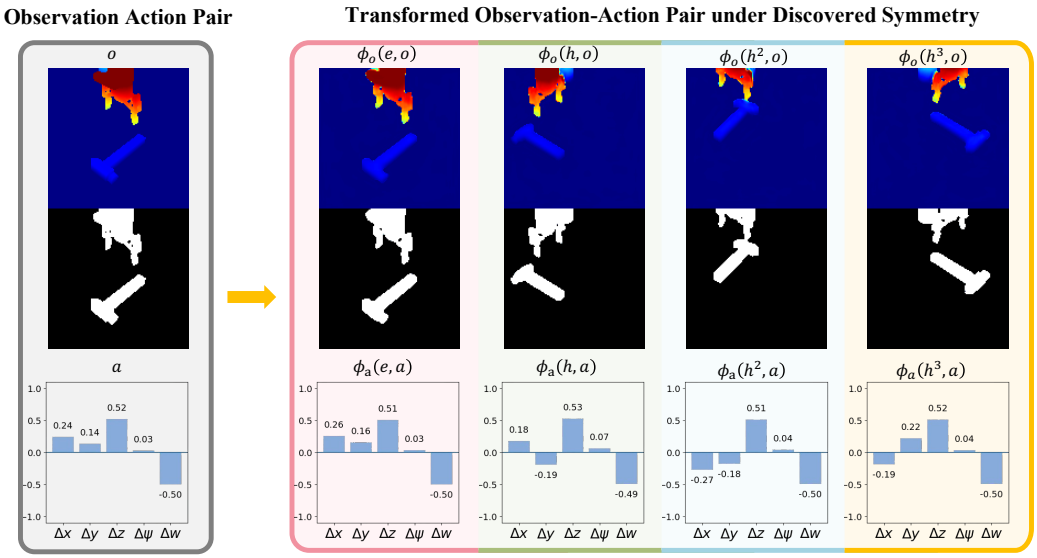


(b) Drawer Closing

Figure 15: **Learned transformations with simulated heightmap and segmentation-mask observations.** Each task panel shows one fixed original pair (o, a) on the left and four transformed pairs on the right for $g \in C_4 = \{e, h, h^2, h^3\}$, with latent angles $0°, 90°, 180°, 270°$, respectively. Within each observation, the heightmap appears above the segmentation mask; action bars below show normalized command components on a $[-1, 1]$ scale. The same g transforms both observation channels and the action jointly; decoded transformations need not be geometric rotations by the latent angles. Panels (c)–(d) continue on the next page.



(c) Cube Dropping



(d) Hammer Lifting

Figure 15: **Learned transformations with simulated heightmap and segmentation-mask observations (continued).** Panels (c)–(d) show Cube Dropping and Hammer Lifting, with the original pair on the left and four transformed pairs on the right. Columns e, h, h^2, h^3 correspond to latent angles $0°, 90°, 180°, 270°$, respectively; decoded transformations need not be geometric rotations by these angles. Each observation shows the heightmap above the segmentation mask, with normalized action components on a $[-1, 1]$ scale below.